OFFICIAL DOCUMENTATION · OFFLINE REFERENCE

ChromaDB Documentation

Official documentation for chromadb 1.5.9 (Python)

SOURCE	docs.trychroma.com, from github.com/chroma-core/chroma at release tag 1.5.9
VERSION	1.5.9 (same as the version pip installs)
COVERS	Clients (persistent), collections, adding/querying data, metadata filters, embedding functions, Ollama embeddings, LangChain integration, retrieval and chunking guides, Python API reference

Index

Page numbers match the page counter at the bottom of each page. Every entry is a clickable link.

Parts at a glance

Overview	2 topics	**5**
Run Chroma	2 topics	**10**
Collections	5 topics	**13**
Querying collections	3 topics	**25**
Embeddings	2 topics	**34**
Integrations	3 topics	**40**
Guides	4 topics	**43**
Python API reference	4 topics	**59**
Other	2 topics	**73**

Detailed contents

Overview 5

1 Introduction 5
- 1.1 What Chroma Offers 5
- 1.2 Quickstart 5
- 1.3 Example Projects 5
- 1.4 Open Source 5

2 Getting Started 7
- 2.1 Install with AI 7
- 2.2 Install Manually 7
- 2.3 Next steps 9

Run Chroma 10

3 Chroma Clients 10
- 3.1 Cloud Client 10
- 3.2 In-Memory Client 10
- 3.3 Persistent Client 11

4 Client-Server Mode 12

Collections 13

5 Manage Collections 13
- 5.1 Creating Collections 13
- 5.2 Getting Collections 14
- 5.3 Modifying Collections 15
- 5.4 Deleting Collections 15
- 5.5 Convenience Methods 15

6 Adding Data to Chroma Collections 16
- 6.1 Adding Data 16
- 6.2 Metadata 16
- 6.3 Behaviors 17

7 Update Data 18
8 Delete Data 19
9 Configure Collections 20
9.1 HNSW Index Configuration 20
9.2 SPANN Index Configuration 22
9.3 Embedding Function Configuration 23

Querying collections 25

10 Query and Get 25
10.1 Query 25
10.2 Get 26
10.3 Results Shape 26
10.4 Choosing Which Data is Returned 27
11 Metadata Filtering 28
11.1 Using Logical Operators 28
11.2 Using Inclusion Operators 29
11.3 Using Array Metadata 30
11.4 Combining with Document Search 31
12 Full Text Search 32
12.1 Using Logical Operators 32
12.2 Combining with Metadata Filtering 32

Embeddings 34

13 Embedding Functions 34
13.1 Using Embedding Functions 34
13.2 Custom Embedding Functions 34
13.3 Default: all-MiniLM-L6-v2 35
13.4 All Embedding Functions 35
14 Multimodal Embeddings 37
14.1 Multi-modal Embedding Functions 37
14.2 Adding Multimodal Data and Data Loaders 37
14.3 Querying 38
14.4 Updating 38

Integrations 40

15 Ollama 40
16 Nomic 41
17 Langchain 42
17.1 Langchain - Python 42
17.2 Langchain - JS 42

Guides 43

18 Intro to Retrieval 43
18.1 The Knowledge Problem 43
18.2 Enter Retrieval 44
19 Chunking 47
19.1 Choosing Chunking Boundaries 47
19.2 Chunking Strategies 48
19.3 Chunking Text 49
19.4 Chunking Code 49

19.5 Evaluation 51

20 Look at Your Data 53
20.1 Search Modalities 53
20.2 Embedding Models 55
20.3 Collections in your Chroma Database 55
20.4 Chunking Data 55
20.5 Metadata 56

21 Building with AI 57

Python API reference 59

22 Client 59
22.1 Clients 59
22.2 Client Methods 60
22.3 Admin Client Methods 62

23 Collection 64
23.1 Collection Methods 64
23.2 Types 67

24 Embedding Functions 69
24.1 Embedding Function Base Classes 69
24.2 Registration 69
24.3 Types 70

25 Where Filters 71
25.1 Field references 71
25.2 Comparison operators 71
25.3 Set operators 71
25.4 Array operators 71
25.5 Document operators 71

Other 73

26 Troubleshooting 73
26.1 Chroma JS-Client failures on NextJS projects 73
26.2 Cannot return the results in a contiguous 2D array. Probably ef or M is too small 73
26.3 Using .get or .query, embeddings say None 73
26.4 Build error when running pip install chromadb 73
26.5 SQLite 74
26.6 Illegal instruction (core dumped) 74
26.7 My data directory is too large 74

27 Migration 75
27.1 Migration Log 75

Introduction

Source: https://docs.trychroma.com/docs/overview/introduction

Chroma gives you everything you need for retrieval: store embeddings with metadata, search with dense and sparse vectors, filter by metadata, and retrieve across text, images, and more.

What Chroma Offers

Document Storage

Store documents and metadata.

Embeddings

Use any embedding model. OpenAI, Cohere, Hugging Face, sentence-transformers, and more.

Vector Search

Dense, sparse, and hybrid search. Query by similarity and combine multiple search strategies.

Full-Text & Regex Search

Keyword and regex search over your data without embeddings.

Metadata Filtering

Filter results at query time by metadata conditions.

Multi-Modal Retrieval

Index and search images, audio, and other modalities alongside text.

Quickstart

Getting Started with the Chroma SDK

Create a self-hosted or cloud database and add data to it using the Chroma SDK.

Create a Chroma Cloud Database

Create a scalable, zero-ops Chroma Cloud database to store your AI data.

Example Projects

Agentic Search

Build agents that iteratively search and refine results for complex queries.

Code Search

Index codebases to power coding agents using AST-aware chunking.

Open Source

Chroma is licensed under Apache 2.0. Run it locally, self-host, or use Chroma Cloud for a managed, serverless experience.

Getting Started

Source: https://docs.trychroma.com/docs/overview/getting-started

Python

> **Note:** For production, Chroma offers Chroma Cloud - a fast, scalable, and serverless database-as-a-service. Get started in 30 seconds - $5 in free credits included.

Install with AI

Give the following prompt to Claude Code, Cursor, Codex, or your favorite AI agent. It will quickly set you up with Chroma.

Chroma Cloud expandable

```
In this directory create a new Python project with Chroma set up.
Use a virtual environment.

Write a small example that adds some data to a collection and queries it.
Do not delete the data from the collection when it's complete.
Run the script when you are done setting up the environment and writing the
script. The output should show what data was ingested, what was the query,
and the results.
Your own summary should include this output so the user can see it.

First, install `chromadb`.

The project should be set up with Chroma Cloud. When you install `chromadb`,
you get access to the Chroma CLI. You can run `chroma login` to authenticate.
This will open a browser for authentication and save a connection profile
locally.

You can also use `chroma profile show` to see if the user already has an
active profile saved locally. If so, you can skip the login step.

Then create a DB using the CLI with `chroma db create chroma-getting-started`.
This will create a DB with this name.

Then use the CLI command `chroma db connect chroma-getting-started --env-file`.
This will create a .env file in the current directory with the connection
variables for this DB and account, so the CloudClient can be instantiated
with chromadb.CloudClient(api_key=os.getenv("CHROMA_API_KEY"), ...).
```

OSS expandable

```
In this directory create a new Python project with Chroma set up.
Use a virtual environment.

Write a small example that adds some data to a collection and queries it.
Do not delete the data from the collection when it's complete.
Run the script when you are done setting up the environment and writing the
script. The output should show what data was ingested, what was the query,
and the results.
Your own summary should include this output so the user can see it.

Use Chroma's in-memory client: `chromadb.Client()`
```

Install Manually

Install

pip

```
pip install chromadb
```

poetry

```
poetry add chromadb
```

uv

```
uv pip install chromadb
```

Create a Chroma Client

Python

```python
import chromadb
chroma_client = chromadb.Client()
```

Create a collection

Collections are where you'll store your embeddings, documents, and any additional metadata. Collections index your embeddings and documents, and enable efficient retrieval and filtering. You can create a collection with a name:

Python

```python
collection = chroma_client.create_collection(name="my_collection")
```

Add some text documents to the collection

Chroma will store your text and handle embedding and indexing automatically. You can also customize the embedding model. You must provide unique string IDs for your documents.

Python

```python
collection.add(
    ids=["id1", "id2"],
    documents=[
        "This is a document about pineapple",
        "This is a document about oranges"
    ]
)
```

Query the collection

You can query the collection with a list of query texts, and Chroma will return the n most similar results. It's that easy!

Python

```python
results = collection.query(
    query_texts=["This is a query document about hawaii"], # Chroma will embed this for you
    n_results=2 # how many results to return
)
print(results)
```

If n_results is not provided, Chroma will return 10 results by default. Here we only added 2 documents, so we set n_results=2.

Inspect Results

From the above - you can see that our query about hawaii is semantically most similar to the document about pineapple.

Python

```
{
  'documents': [[
      'This is a document about pineapple',
      'This is a document about oranges'
  ]],
  'ids': [['id1', 'id2']],
  'distances': [[1.0404009819030762, 1.243080496788025]],
  'uris': None,
  'data': None,
  'metadatas': [[None, None]],
  'embeddings': None,
}
```

Try it out yourself

What if we tried querying with "This is a document about florida"? Here is a full example.

Python expandable

```
import chromadb
chroma_client = chromadb.Client()

# switch \`create_collection\` to \`get_or_create_collection\` to avoid creating a new
collection every time
collection = chroma_client.get_or_create_collection(name="my_collection")

# switch \`add\` to \`upsert\` to avoid adding the same documents every time
collection.upsert(
    documents=[
        "This is a document about pineapple",
        "This is a document about oranges"
    ],
    ids=["id1", "id2"]
)

results = collection.query(
    query_texts=["This is a query document about florida"], # Chroma will embed this for you
    n_results=2 # how many results to return
)

print(results)
```

Next steps

In this guide we used Chroma's in-memory client for simplicity. It starts a Chroma server in-memory, so any data you ingest will be lost when your program terminates. You can use the persistent client or run Chroma in client-server mode if you need data persistence.

- Learn how to Deploy Chroma to a server
- Join Chroma's Discord Community to ask questions and get help
- Follow Chroma on X (@trychroma) for updates

Chroma Clients

Source: https://docs.trychroma.com/docs/run-chroma/clients

There are several ways you can instantiate clients to connect to your Chroma database.

Cloud Client

You can use the `CloudClient` to create a client connecting to Chroma Cloud.

Python

```
client = chromadb.CloudClient(
    tenant='Tenant ID',
    database='Database name',
    api_key='Chroma Cloud API key'
)
```

The `CloudClient` can be instantiated just with the API key argument. In which case, we will resolve the tenant and DB from Chroma Cloud. Note our auto-resolution will work only if the provided API key is scoped to a single DB.

If you set the `CHROMA_API_KEY`, `CHROMA_TENANT`, and the `CHROMA_DATABASE` environment variables, you can simply instantiate a `CloudClient` with no arguments:

Python

```
client = chromadb.CloudClient()
```

Connecting to a non-default region

By default, `CloudClient` connects to `api.trychroma.com` (AWS `us-east-1`). To target a database in another region — for example, our GCP `europe-west1` region — point the client at that region's hostname. See Regions for the list of available endpoints.

Python

```
client = chromadb.CloudClient(
    cloud_host='europe-west1.gcp.trychroma.com',
    cloud_port=443,
    api_key='Chroma Cloud API key',
    tenant='Tenant ID',
    database='Database name',
)
```

The dashboard's **Connect** panel generates a ready-to-paste snippet for whichever region a database lives in, including the corresponding `.env` block. For databases outside `aws-us-east-1`, the snippet sets `CHROMA_HOST` to the region-specific host:

```
CHROMA_HOST=europe-west1.gcp.trychroma.com
CHROMA_API_KEY=...
CHROMA_TENANT=...
CHROMA_DATABASE=...
```

In-Memory Client

In Python, you can run a Chroma server in-memory and connect to it with the ephemeral client:

```
client = chromadb.Client()
```

The `Client()` method starts a Chroma server in-memory and also returns a client with which you can connect to it.

This is a great tool for experimenting with different embedding functions and retrieval techniques in a Python notebook, for example. If you don't need data persistence, the ephemeral client is a good choice for getting up and running with Chroma.

Persistent Client

Python

You can configure Chroma to save and load the database from your local machine, using the `PersistentClient`.

Data will be persisted automatically and loaded on start (if it exists).

```
client = chromadb.PersistentClient(path="/path/to/save/to")
```

The `path` is where Chroma will store its database files on disk, and load them on start. If you don't provide a path, the default is `.chroma`

The client object has a few useful convenience methods.

- `heartbeat()` - returns a nanosecond heartbeat. Useful for making sure the client remains connected.
- `reset()` - empties and completely resets the database. WARNING: This is destructive and not reversible.

```
client.heartbeat()
client.reset()
```

Client-Server Mode

Source: https://docs.trychroma.com/docs/run-chroma/client-server

Chroma can also be configured to run in client/server mode. In this mode, the Chroma client connects to a Chroma server running in a separate process.

To start the Chroma server, run the following command:

```
chroma run --path /db_path
```

Python

Then use the Chroma `HttpClient` to connect to the server:

```
chroma_client = chromadb.HttpClient(host='localhost', port=8000)
```

That's it! Chroma's API will run in `client-server` mode with just this change.

Chroma also provides the async HTTP client. The behaviors and method signatures are identical to the synchronous client, but all methods that would block are now async. To use it, call `AsyncHttpClient` instead:

```
async def main():
    client = await chromadb.AsyncHttpClient()

    collection = await client.create_collection(name="my_collection")
    await collection.add(
        documents=["hello world"],
        ids=["id1"]
    )

asyncio.run(main())
```

If you deploy your Chroma server, you can also use our http-only package.

Manage Collections

Source: https://docs.trychroma.com/docs/collections/manage-collections

Chroma lets you manage collections of embeddings, using the **collection** primitive. Collections are the fundamental unit of storage and querying in Chroma.

Creating Collections

Chroma collections are created with a name. Collection names are used in the url, so there are a few restrictions on them:

- The length of the name must be between 3 and 512 characters.
- The name must start and end with a lowercase letter or a digit, and it can contain dots, dashes, and underscores in between.
- The name must not contain two consecutive dots.
- The name must not be a valid IP address.

Python

```
collection = client.create_collection(name="my_collection")
```

Note that collection names must be **unique** inside a Chroma database. If you try to create a collection with a name of an existing one, you will see an exception.

Embedding Functions

When you add documents to a collection, Chroma will embed them for you by using the collection's **embedding function**. Chroma will use sentence transformer embedding function as a default.

Chroma also offers various embedding function, which you can provide upon creating a collection. For example, you can create a collection using the `OpenAIEmbeddingFunction`:

Python

Install the `openai` package:

pip

```
pip install openai
```

poetry

```
poetry add openai
```

uv

```
uv pip install openai
```

Create your collection with the `OpenAIEmbeddingFunction`:

```
from chromadb.utils.embedding_functions import OpenAIEmbeddingFunction

collection = client.create_collection(
    name="my_collection",
```

```
    embedding_function=OpenAIEmbeddingFunction(
        api_key=os.getenv("OPENAI_API_KEY"),
        model_name="text-embedding-3-small"
    )
)
```

Instead of having Chroma embed documents, you can also provide embeddings directly when adding data to a collection. In this case, your collection will not have an embedding function set, and you will be responsible for providing embeddings directly when adding data and querying.

```
collection = client.create_collection(
    name="my_collection",
    embedding_function=None
)
```

Collection Metadata

When creating collections, you can pass the optional `metadata` argument to add a mapping of metadata key-value pairs to your collections. This can be useful for adding general information about the collection like creation time, description of the data stored in the collection, and more.

Python

```
from datetime import datetime

collection = client.create_collection(
    name="my_collection",
    embedding_function=emb_fn,
    metadata={
        "description": "my first Chroma collection",
        "created": str(datetime.now())
    }
)
```

Getting Collections

Python

There are several ways to get a collection after it was created.

The `get_collection` function will get a collection from Chroma by name. It returns a `Collection` object with `name`, `metadata`, `configuration`, and `embedding_function`.

```
collection = client.get_collection(name="my-collection")
```

The `get_or_create_collection` function behaves similarly, but will create the collection if it doesn't exist. You can pass to it the same arguments `create_collection` expects, and the client will ignore them if the collection already exists.

```
collection = client.get_or_create_collection(
    name="my-collection",
    metadata={"description": "..."}
)
```

The `list_collections` function returns the collections you have in your Chroma database. The collections will be ordered by creation time from oldest to newest.

```
collections = client.list_collections()
```

By default, `list_collections` returns up to 100 collections. If you have more than 100 collections, or need to get only a subset of your collections, you can use the `limit` and `offset` arguments:

```
first_collections_batch = client.list_collections(limit=100) # get the first 100 collections
second_collections_batch = client.list_collections(limit=100, offset=100) # get the next 100 collections
collections_subset = client.list_collections(limit=20, offset=50) # get 20 collections starting from the 50th
```

Current versions of Chroma store the embedding function you used to create a collection on the server, so the client can resolve it for you on subsequent "get" operations. If you are running an older version of the Chroma client or server (earlier than 1.1.13), you will need to provide the same embedding function you used to create a collection when using `get_collection`:

```
collection = client.get_collection(
    name='my-collection',
    embedding_function=ef
)
```

Modifying Collections

After a collection is created, you can modify its name, metadata and elements of its index configuration with the `modify` method:

Python

```
collection.modify(
   name="new-name",
   metadata={"description": "new description"}
)
```

Deleting Collections

You can delete a collection by name. This action will delete a collection, all of its embeddings, and associated documents and records' metadata.

> **Warning:** Deleting collections is destructive and not reversible

Python

```
client.delete_collection(name="my-collection")
```

Convenience Methods

Collections also offer a few useful convenience methods:

- `count` - returns the number of records in the collection.
- `peek` - returns the first 10 records in the collection.

Python

```
collection.count()
collection.peek()
```

Adding Data to Chroma Collections

Source: https://docs.trychroma.com/docs/collections/add-data

Adding Data

Use `.add` to insert new records into a collection. Each record needs a unique string `id`.

Python

```
collection.add(
    ids=["id1", "id2", "id3"],
    documents=["lorem ipsum...", "doc2", "doc3"],
    metadatas=[{"chapter": 3, "verse": 16}, {"chapter": 3, "verse": 5}, {"chapter": 29,
"verse": 11}],
)
```

You must provide either `documents`, `embeddings`, or both. `metadatas` are always optional. When only providing `documents`, Chroma will generate embeddings for you using the collection's embedding function.

If you've already computed embeddings, pass them alongside `documents`. Chroma will store both as-is without re-embedding the documents.

Python

```
collection.add(
    ids=["id1", "id2", "id3"],
    embeddings=[[1.1, 2.3, 3.2], [4.5, 6.9, 4.4], [1.1, 2.3, 3.2]],
    documents=["doc1", "doc2", "doc3"],
    metadatas=[{"chapter": 3, "verse": 16}, {"chapter": 3, "verse": 5}, {"chapter": 29,
"verse": 11}],
)
```

If your documents are stored elsewhere, you can add just embeddings and metadata. Use the `ids` to associate records with your external documents. This is a useful pattern if your documents are very large, such as high-resolution images or videos.

Python

```
collection.add(
    ids=["id1", "id2", "id3"],
    embeddings=[[1.1, 2.3, 3.2], [4.5, 6.9, 4.4], [1.1, 2.3, 3.2]],
    metadatas=[{"chapter": 3, "verse": 16}, {"chapter": 3, "verse": 5}, {"chapter": 29,
"verse": 11}],
)
```

Metadata

Metadata values can be strings, integers, floats, or booleans. Additionally, you can store arrays of these types.

Python

```
collection.add(
    ids=["id1"],
    documents=["lorem ipsum..."],
    metadatas=[{
        "chapter": 3,
        "tags": ["fiction", "adventure"],
        "scores": [1, 2, 3],
```

```
    }],
)
```

All elements in an array must be the same type, and empty arrays are not allowed. You can filter on array metadata using the `$contains` and `$not_contains` operators — see Metadata Filtering for details.

Behaviors

- If you add a record with an ID that already exists in the collection, it will be ignored without throwing an error. In order to overwrite data in your collection, you must update the data.
- If the supplied embeddings don't match the dimensionality of embeddings already in the collection, an exception will be raised.

Update Data

Source: https://docs.trychroma.com/docs/collections/update-data

Any property of records in a collection can be updated with `.update`:

Python

```
collection.update(
    ids=["id1", "id2", "id3", ...],
    embeddings=[[1.1, 2.3, 3.2], [4.5, 6.9, 4.4], [1.1, 2.3, 3.2], ...],
    metadatas=[{"chapter": 3, "verse": 16}, {"chapter": 3, "verse": 5}, {"chapter": 29,
"verse": 11}, ...],
    documents=["doc1", "doc2", "doc3", ...],
)
```

If an `id` is not found in the collection, an error will be logged and the update will be ignored. If `documents` are supplied without corresponding `embeddings`, the embeddings will be recomputed with the collection's embedding function.

Metadata values can include arrays — see Adding Data for supported metadata types.

If the supplied `embeddings` are not the same dimension as the collection, an exception will be raised.

Chroma also supports an `upsert` operation, which updates existing items, or adds them if they don't yet exist.

Python

```
collection.upsert(
    ids=["id1", "id2", "id3", ...],
    embeddings=[[1.1, 2.3, 3.2], [4.5, 6.9, 4.4], [1.1, 2.3, 3.2], ...],
    metadatas=[{"chapter": 3, "verse": 16}, {"chapter": 3, "verse": 5}, {"chapter": 29,
"verse": 11}, ...],
    documents=["doc1", "doc2", "doc3", ...],
)
```

If an `id` is not present in the collection, the corresponding items will be created as per `add`. Items with existing `id`s will be updated as per `update`.

Delete Data

Source: https://docs.trychroma.com/docs/collections/delete-data

Chroma supports deleting items from a collection by `id` using `.delete`. The embeddings, documents, and metadata associated with each item will be deleted.

> **Warning:** Naturally, this is a destructive operation, and cannot be undone.

Python

```
collection.delete(
    ids=["id1", "id2", "id3",...],
)
```

`.delete` also supports the `where` filter. It will delete all items in the collection that match the `where` filter.

Python

```
collection.delete(
    where={"chapter": "20"}
)
```

Configure Collections

Source: https://docs.trychroma.com/docs/collections/configure

Chroma collections have a `configuration` that determines how their embeddings index is constructed and used. We use default values for these index configurations that should give you great performance for most use cases out-of-the-box.

The embedding function you choose to use in your collection also affects its index construction, and is included in the configuration.

When you create a collection, you can customize these index configuration values for different data, accuracy and performance requirements. Some query-time configurations can also be customized after the collection's creation using the `.modify` function.

Single Node

HNSW Index Configuration

In Single Node Chroma collections, we use an HNSW (Hierarchical Navigable Small World) index to perform approximate nearest neighbor (ANN) search.

What is an HNSW index?

An HNSW (Hierarchical Navigable Small World) index is a graph-based data structure designed for efficient approximate nearest neighbor search in high-dimensional vector spaces. It works by constructing a multi-layered graph where each layer contains a subset of the data points, with higher layers being sparser and serving as "highways" for faster navigation. The algorithm builds connections between nearby points at each layer, creating "small-world" properties that allow for efficient search complexity. During search, the algorithm starts at the top layer and navigates toward the query point in the embedding space, then moves down through successive layers, refining the search at each level until it finds the final nearest neighbors.

The HNSW index parameters include:

- `space` defines the distance function of the embedding space, and hence how similarity is defined. The default is `l2` (squared L2 norm), and other possible values are `cosine` (cosine similarity), and `ip` (inner product).

Distance	parameter	Equation	Intuition
Squared L2	`l2`	$d = \sum\left(A_i-B_i\right)^2$	measures absolute geometric distance between vectors, making it suitable when you want true spatial proximity.
Inner product	`ip`	$d = 1.0 - \sum\left(A_i \times B_i\right)$	focuses on vector alignment and magnitude, often used for recommendation systems where larger values indicate stronger preferences
Cosine similarity	`cosine`	$d = 1.0 - \frac{\sum\left(A_i \times B_i\right)}{\sqrt{\sum\left(A_i^2\right)} \cdot \sqrt{\sum\left(B_i^2\right)}}$	measures only the angle between vectors (ignoring magnitude), making it ideal for text embeddings or cases where you care about direction rather than scale

> **Warning:** You should make sure that the `space` you choose is supported by your collection's embedding function. Every Chroma embedding function specifies its default space and a list of supported spaces.

- `ef_construction` determines the size of the candidate list used to select neighbors during index creation. A higher value improves index quality at the cost of more memory and time, while a lower value speeds up construction with reduced accuracy. The default value is `100`.
- `ef_search` determines the size of the dynamic candidate list used while searching for the nearest neighbors. A higher value improves recall and accuracy by exploring more potential neighbors but increases query time and computational cost, while a lower value results in faster but less accurate searches. The default value is `100`. This field can be modified after creation.
- `max_neighbors` is the maximum number of neighbors (connections) that each node in the graph can have during the construction of the index. A higher value results in a denser graph, leading to better recall and accuracy during searches but increases memory usage and construction time. A lower value creates a sparser graph, reducing memory usage and construction time but at the cost of lower search accuracy and recall. The default value is `16`.
- `num_threads` specifies the number of threads to use during index construction or search operations. The default value is `multiprocessing.cpu_count()` (available CPU cores). This field can be modified after creation.
- `batch_size` controls the number of vectors to process in each batch during index operations. The default value is `100`. This field can be modified after creation.
- `sync_threshold` determines when to synchronize the index with persistent storage. The default value is `1000`. This field can be modified after creation.
- `resize_factor` controls how much the index grows when it needs to be resized. The default value is `1.2`. This field can be modified after creation.

For example, here we create a collection with customized values for `space` and `ef_construction`:

Python

```
collection = client.create_collection(
    name="my-collection",
    embedding_function=OpenAIEmbeddingFunction(model_name="text-embedding-3-small"),
    configuration={
        "hnsw": {
            "space": "cosine",
            "ef_construction": 200
        }
    }
)
```

Fine-Tuning HNSW Parameters

In the context of approximate nearest neighbors search, **recall** refers to how many of the true nearest neighbors were retrieved.

Increasing `ef_search` normally improves recall, but slows down query time. Similarly, increasing `ef_construction` improves recall, but increases the memory usage and runtime when creating the index.

Choosing the right values for your HNSW parameters depends on your data, embedding function, and requirements for recall, and performance. You may need to experiment with different

construction and search values to find the values that meet your requirements.

For example, for a dataset with 50,000 embeddings of 2048 dimensions, generated by

```
embeddings = np.random.randn(50000, 2048).astype(np.float32).tolist()
```

we set up two Chroma collections:

- The first is configured with `ef_search: 10`. When querying using a specific embedding from the set (with `id = 1`), the query takes `0.00529` seconds, and we get back embeddings with distances:

```
[3629.019775390625, 3666.576904296875, 3684.57080078125]
```

- The second collection is configured with `ef_search: 100` and `ef_construction: 1000`. When issuing the same query, this time it takes `0.00753` seconds (about 42% slower), but with better results as measured by their distance:

```
[0.0, 3620.593994140625, 3623.275390625]
```

In this example, when querying with the test embedding (`id=1`), the first collection failed to find the embedding itself, despite it being in the collection (where it should have appeared as a result with a distance of `0.0`). The second collection, while slightly slower, successfully found the query embedding itself (shown by the `0.0` distance) and returned closer neighbors overall, demonstrating better accuracy at the cost of performance.

Distributed and Chroma Cloud

SPANN Index Configuration

In Distributed Chroma and Chroma Cloud collections, we use a SPANN (Spacial Approximate Nearest Neighbors) index to perform approximate nearest neighbor (ANN) search.

What is a SPANN index?

A SPANN index is a data structure used to efficiently find approximate nearest neighbors in large sets of high-dimensional vectors. It works by dividing the set into broad clusters (so we can ignore most of the data during search) and then building efficient, smaller indexes within each cluster for fast local lookups. This two-level approach helps reduce both memory use and search time, making it practical to search billions of vectors stored even on hard drives or separate machines in a distributed system.

> **Warning:** We currently don't allow customization or modification of SPANN configuration. If you set these values they will be ignored by the server.

The SPANN index parameters include:

- `space` defines the distance function of the embedding space, and hence how similarity is defined. The default is `l2` (squared L2 norm), and other possible values are `cosine` (cosine similarity), and `ip` (inner product).

Distance	parameter	Equation	Intuition
Squared L2	`l2`	$d = \sum\left(A_i-B_i\right)^2$	measures absolute geometric distance between vectors, making it suitable when you want true spatial proximity.
Inner product	`ip`	$d = 1.0 - \sum\left(A_i \times B_i\right)$	focuses on vector alignment and magnitude, often used for recommendation systems where larger values indicate stronger preferences
Cosine similarity	`cosine`	$d = 1.0 - \frac{\sum\left(A_i \times B_i\right)}{\sqrt{\sum\left(A_i^2\right)} \cdot \sqrt{\sum\left(B_i^2\right)}}$	measures only the angle between vectors (ignoring magnitude), making it ideal for text embeddings or cases where you care about direction rather than scale

- `search_nprobe` is the number of centers that are probed for a query. The higher the value the more accurate the result will be. The query response time also increases as `search_nprobe` increases. Recommended values are 64/128. We don't allow setting a value higher than 128 today. The default value is 64.
- `write_nprobe` is the same as `search_nprobe` but for the index construction phase. It is the number of centers searched when appending or reassigning a point. It has the same limits as `search_nprobe`. The default value is 64.
- `ef_construction` determines the size of the candidate list used to select neighbors during index creation. A higher value improves index quality at the cost of more memory and time, while a lower value speeds up construction with reduced accuracy. The default value is 200.
- `ef_search` determines the size of the dynamic candidate list used while searching for the nearest neighbors. A higher value improves recall and accuracy by exploring more potential neighbors but increases query time and computational cost, while a lower value results in faster but less accurate searches. The default value is 200.
- `max_neighbors` defines the maximum number of neighbors for a node. The default value is 64.
- `reassign_neighbor_count` is the number of closest neighboring clusters of a split cluster whose points are considered for reassignment. The default value is 64.

Embedding Function Configuration

The embedding function you choose when creating a collection, along with the parameters you instantiate it with, is persisted in the collection's configuration. This allows us to reconstruct it correctly when you use collection across different clients.

You can set your embedding function as an argument to the "create" methods, or directly in the configuration:

Python

Install the `openai` and `cohere` packages:

pip

```
pip install openai cohere
```

poetry

```
poetry add openai cohere
```

uv

```
uv pip install openai cohere
```

Creating collections with embedding function and custom configuration:

```
from chromadb.utils.embedding_functions import OpenAIEmbeddingFunction,
CohereEmbeddingFunction

# Using the `embedding_function` argument
openai_collection = client.create_collection(
    name="my_openai_collection",
    embedding_function=OpenAIEmbeddingFunction(
        model_name="text-embedding-3-small"
    ),
    configuration={"hnsw": {"space": "cosine"}}
)

# Setting `embedding_function` in the collection's `configuration`
cohere_collection = client.get_or_create_collection(
    name="my_cohere_collection",
    configuration={
        "embedding_function": CohereEmbeddingFunction(
            model_name="embed-english-light-v2.0",
            truncate="NONE"
        ),
        "hnsw": {"space": "cosine"}
    }
)
```

Note: Many embedding functions require API keys to interface with the third party embeddings providers. The Chroma embedding functions will automatically look for the standard environment variable used to store a provider's API key. For example, the Chroma `OpenAIEmbeddingFunction` will set its `api_key` argument to the value of the `OPENAI_API_KEY` environment variable if it is set.

If your API key is stored in an environment variable with a non-standard name, you can configure your embedding function to use your custom environment variable by setting the `api_key_env_var` argument. In order for the embedding function to operate correctly, you will have to set this variable in every environment where you use your collection.

```
cohere_ef = CohereEmbeddingFunction(
    api_key_env_var="MY_CUSTOM_COHERE_API_KEY",
    model_name="embed-english-light-v2.0",
    truncate="NONE",
)
```

Query and Get

Source: https://docs.trychroma.com/docs/querying-collections/query-and-get

> **Note:** Dense vector search, hybrid search, and more are available in the new powerful Search API for Chroma Cloud databases.

The Query API enables nearest-neighbor similarity search over dense embeddings. Use the Get API when you want to retrieve records without similarity ranking.

Python

Query

You can query a collection to run a similarity search using `.query`:

```
collection.query(
    query_texts=["thus spake zarathustra", "the oracle speaks"]
)
```

Chroma will use the collection's embedding function to embed your text queries, and use the output to run a vector similarity search against your collection.

Instead of providing `query_texts`, you can provide `query_embeddings` directly. You will be required to do so if your collection does not have an embedding function attached to it. The dimension of your query embedding must match the dimension of the embeddings in your collection.

Python also supports `query_images` and `query_uris` as query inputs.

```
collection.query(
    query_embeddings=[[11.1, 12.1, 13.1], [1.1, 2.3, 3.2]]
)
```

By default, Chroma will return 10 results per input query. You can modify this number using the `n_results` argument:

```
collection.query(
    query_embeddings=[[11.1, 12.1, 13.1], [1.1, 2.3, 3.2]],
    n_results=100
)
```

The `ids` argument lets you constrain the search only to records with the IDs from the provided list:

```
collection.query(
    query_embeddings=[[11.1, 12.1, 13.1], [1.1, 2.3, 3.2]],
    n_results=100,
    ids=["id1", "id2"]
)
```

Both `query` and `get` support `where` for metadata filtering and `where_document` for full-text search and regex:

```
collection.query(
    query_embeddings=[[11.1, 12.1, 13.1], [1.1, 2.3, 3.2]],
    n_results=100,
```

```
    where={"page": 10}, # query records with metadata field 'page' equal to 10
    where_document={"$contains": "search string"} # query records with the search string in
the records' document
)
```

Get

Use `.get` to retrieve records by ID and/or filters without similarity ranking:

```
collection.get(ids=["id1", "id2"]) # by IDs

collection.get(limit=100, offset=0) # with pagination
```

Results Shape

Chroma returns `.query` and `.get` results in **column-major** form (arrays per field). `.query` results are grouped per input query; `.get` results are a flat list of records.

Python

```
class QueryResult(TypedDict):
    ids: List[IDs]
    embeddings: Optional[List[Embeddings]]
    documents: Optional[List[List[Document]]]
    uris: Optional[List[List[URI]]]
    metadatas: Optional[List[List[Metadata]]]
    distances: Optional[List[List[float]]]
    included: Include

class GetResult(TypedDict):
    ids: List[ID]
    embeddings: Optional[Embeddings]
    documents: Optional[List[Document]]
    uris: Optional[URIs]
    metadatas: Optional[List[Metadata]]
    included: Include
```

Here is a concrete example of what these responses look like in practice:

```
// Query result
{
  "ids": [["doc_1", "doc_7"]],
  "embeddings": [[[1, 2, 3, 4], [1, 2, 3, 4]]],
  "documents": [["Chroma stores vectors.", "Embeddings power semantic search."]],
  "metadatas": [[
    {"source": "docs", "topic": "intro"},
    {"source": "blog", "topic": "search"}
  ]],
  "distances": [[0.12, 0.21]],
  "included": ["embeddings", "documents", "metadatas", "distances"]
}
// Get result
{
  "ids": ["doc_1", "doc_7"],
  "embeddings": [[1, 2, 3, 4], [1, 2, 3, 4]],
  "documents": ["Chroma stores vectors.", "Embeddings power semantic search."],
  "metadatas": [
    {"source": "docs", "topic": "intro"},
    {"source": "blog", "topic": "search"}
  ],
  "included": ["documents", "metadatas"]
}
```

In the results from the Get operation, corresponding elements in each array belong to the same document.

Python

```python
result = collection.get(include=["documents", "metadatas"])
for id, document, metadata in zip(result["ids"], result["documents"], result["metadatas"]):
    print(id, document, metadata)
```

Query is a batch API and returns results grouped per input. A common pattern is to iterate over each query's “batch” of results, then iterate within that batch.

Python

```python
result = collection.query(query_texts=["first query", "second query"])
for ids, documents, metadatas in zip(result["ids"], result["documents"],
result["metadatas"]):
    for id, document, metadata in zip(ids, documents, metadatas):
        print(id, document, metadata)
```

Choosing Which Data is Returned

By default, Query returns `documents`, `metadatas`, and `distances`, and Get returns `documents` and `metadatas`.

Use `include` to control what comes back. `ids` are always returned.

Python

```python
collection.query(
    query_texts=["my query"],
    include=["documents", "metadatas", "embeddings"],
)

collection.get(include=["documents"])
```

Metadata Filtering

Source: https://docs.trychroma.com/docs/querying-collections/metadata-filtering

The `where` argument in `get` and `query` is used to filter records by their metadata. For example, in this `query` operation, Chroma will only query records that have the `page` metadata field with the value `10`:

Python

```
collection.query(
    query_texts=["first query", "second query"],
    where={"page": 10}
)
```

In order to filter on metadata, you must supply a `where` filter dictionary to the query. The dictionary must have the following structure:

Python

```
{
    "metadata_field": {
        <Operator>: <Value>
    }
}
```

Using the `$eq` operator is equivalent to using the metadata field directly in your `where` filter.

Python

```
{
    "metadata_field": "search_string"
}

# is equivalent to

{
    "metadata_field": {
        "$eq": "search_string"
    }
}
```

For example, here we query all records whose `page` metadata field is greater than 10:

Python

```
collection.query(
    query_texts=["first query", "second query"],
    where={"page": { "$gt": 10 }}
)
```

Using Logical Operators

You can also use the logical operators `$and` and `$or` to combine multiple filters.

An `$and` operator will return results that match all the filters in the list.

Python

```
{
    "$and": [
        {
```

```
            "metadata_field": {
                <Operator>: <Value>
            }
        },
        {
            "metadata_field": {
                <Operator>: <Value>
            }
        }
    ]
}
```

For example, here we query all records whose `page` metadata field is between 5 and 10:

Python

```
collection.query(
    query_texts=["first query", "second query"],
    where={
        "$and": [
            {"page": {"$gte": 5 }},
            {"page": {"$lte": 10 }},
        ]
    }
)
```

An `$or` operator will return results that match any of the filters in the list.

Python

```
{
    "$or": [
        {
            "metadata_field": {
                <Operator>: <Value>
            }
        },
        {
            "metadata_field": {
                <Operator>: <Value>
            }
        }
    ]
}
```

For example, here we get all records whose `color` metadata field is `red` or `blue`:

Python

```
collection.get(
    where={
        "$or": [
            {"color": "red"},
            {"color": "blue"},
        ]
    }
)
```

Using Inclusion Operators

The following inclusion operators are supported:

- `$in` - a value is in predefined list (string, int, float, bool)
- `$nin` - a value is not in predefined list (string, int, float, bool)

An `$in` operator will return results where the metadata attribute is part of a provided list:

Python

```
{
  "metadata_field": {
    "$in": ["value1", "value2", "value3"]
  }
}
```

An `$nin` operator will return results where the metadata attribute is not part of a provided list (or the attribute's key is not present):

Python

```
{
  "metadata_field": {
    "$nin": ["value1", "value2", "value3"]
  }
}
```

For example, here we get all records whose `author` metadata field is in a list of possible values:

Python

```
collection.get(
    where={
       "author": {"$in": ["Rowling", "Fitzgerald", "Herbert"]}
    }
)
```

Using Array Metadata

Chroma supports storing arrays of values in metadata fields. You can use the `$contains` and `$not_contains` operators to filter records based on whether an array field includes a specific value.

Adding Array Metadata

Metadata arrays can contain strings, integers, floats, or booleans. All elements in an array must be the same type.

Python

```
collection.add(
    ids=["m1", "m2", "m3"],
    embeddings=[[1, 0, 0], [0, 1, 0], [0, 0, 1]],
    metadatas=[
        {"genres": ["action", "comedy"], "year": 2020},
        {"genres": ["drama"], "year": 2021},
        {"genres": ["action", "thriller"], "year": 2022},
    ],
)
```

Filtering with `$contains` and `$not_contains`

Use `$contains` to check if a metadata array includes a specific scalar value, and `$not_contains` to check that it does not.

Python

```
# Get all records where genres contains "action"
collection.get(
    where={"genres": {"$contains": "action"}}
)
```

```
# Get all records where genres does NOT contain "action"
collection.get(
    where={"genres": {"$not_contains": "action"}}
)

# Works with integer arrays too
collection.get(
    where={"scores": {"$contains": 20}}
)

# Combine with other filters
collection.get(
    where={
        "$and": [
            {"genres": {"$contains": "action"}},
            {"year": {"$gte": 2021}},
        ]
    }
)
```

Supported Array Types

Type	Python	TypeScript	Rust
String	`["a", "b"]`	`["a", "b"]`	`MetadataValue::StringArray(...)`
Integer	`[1, 2, 3]`	`[1, 2, 3]`	`MetadataValue::IntArray(...)`
Float	`[1.5, 2.5]`	`[1.5, 2.5]`	`MetadataValue::FloatArray(...)`
Boolean	`[true, false]`	`[true, false]`	`MetadataValue::BoolArray(...)`

Constraints:

- All elements in an array must be the same type.
- Empty arrays are not allowed.
- Nested arrays (arrays of arrays) are not supported.
- The `$contains` value must be a scalar that matches the array's element type.

Combining with Document Search

`.get` and `.query` can handle metadata filtering combined with document search:

Python

```
collection.query(
    query_texts=["doc10", "thus spake zarathustra", ...],
    n_results=10,
    where={"metadata_field": "is_equal_to_this"},
    where_document={"$contains":"search_string"}
)
```

Full Text Search

Source: https://docs.trychroma.com/docs/querying-collections/full-text-search

Python

The `where_document` argument in `get` and `query` is used to filter records based on their document content.

We support full-text search with the `$contains` and `$not_contains` operators. We also support regular expression pattern matching with the `$regex` and `$not_regex` operators.

For example, here we get all records whose document contains a search string:

```
collection.get(
   where_document={"$contains": "search string"}
)
```

Note: Full-text search is case-sensitive.

Here we get all records whose documents match the regex pattern for an email address:

```
collection.get(
   where_document={
       "$regex": "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"
   }
)
```

Using Logical Operators

You can also use the logical operators `$and` and `$or` to combine multiple filters.

An `$and` operator will return results that match all the filters in the list:

```
collection.query(
    query_texts=["query1", "query2"],
    where_document={
        "$and": [
            {"$contains": "search_string_1"},
            {"$regex": "[a-z]+"},
        ]
    }
)
```

An `$or` operator will return results that match any of the filters in the list:

```
collection.query(
    query_texts=["query1", "query2"],
    where_document={
        "$or": [
            {"$contains": "search_string_1"},
            {"$not_contains": "search_string_2"},
        ]
    }
)
```

Combining with Metadata Filtering

`.get` and `.query` can handle `where_document` search combined with metadata filtering:

```
collection.query(
    query_texts=["doc10", "thus spake zarathustra", ...],
    n_results=10,
    where={"metadata_field": "is_equal_to_this"},
    where_document={"$contains":"search_string"}
)
```

Embedding Functions

Source: https://docs.trychroma.com/docs/embeddings/embedding-functions

Embeddings are numeric representations of your data that capture meaning in a form AI models can work with. They can represent text, images, and eventually audio and video. Chroma stores and indexes embeddings so you can efficiently search for similar content. You can generate them locally with an installed library or remotely through an API.

Python

Using Embedding Functions

Embedding functions can be linked to a collection and used whenever you call `add`, `update`, `upsert` or `query`.

For example, this is how you use the OpenAI embedding function:

```
# Set your OPENAI_API_KEY environment variable
from chromadb.utils.embedding_functions import OpenAIEmbeddingFunction

collection = client.create_collection(
    name="my_collection",
    embedding_function=OpenAIEmbeddingFunction(
        model_name="text-embedding-3-small"
    )
)

# Chroma will use OpenAIEmbeddingFunction to embed your documents
collection.add(
    ids=["id1", "id2"],
    documents=["doc1", "doc2"]
)
```

You can also use embedding functions directly which can be handy for debugging.

```
from chromadb.utils.embedding_functions import DefaultEmbeddingFunction

default_ef = DefaultEmbeddingFunction()
embeddings = default_ef(["foo"])
print(embeddings) # [[0.05035809800028801, 0.0626462921500206, -0.061827320605516434...]]

collection.query(query_embeddings=embeddings)
```

Custom Embedding Functions

You can create your own embedding function to use with Chroma; it just needs to implement `EmbeddingFunction`.

```
from typing import Dict, Any
from chromadb import Documents, EmbeddingFunction, Embeddings
from chromadb.utils.embedding_functions import register_embedding_function

@register_embedding_function
class MyEmbeddingFunction(EmbeddingFunction):

    def __init__(self, model):
        self.model = model

    def __call__(self, input: Documents) -> Embeddings:
```

```
        # embed the documents somehow
        return embeddings

    @staticmethod
    def name() -> str:
        return "my-ef"

    def get_config(self) -> Dict[str, Any]:
        return dict(model=self.model)

    @staticmethod
    def build_from_config(config: Dict[str, Any]) -> "EmbeddingFunction":
        return MyEmbeddingFunction(config['model'])
```

Default: all-MiniLM-L6-v2

Chroma's default embedding function uses the Sentence Transformers `all-MiniLM-L6-v2` model to create embeddings. This embedding model can create sentence and document embeddings that can be used for a wide variety of tasks. This embedding function runs locally on your machine, and may require you to download the model files (this will happen automatically).

If you don't specify an embedding function when creating a collection, Chroma will set it to be the `DefaultEmbeddingFunction`:

```
collection = client.create_collection(name="my_collection")
```

All Embedding Functions

Chroma provides lightweight wrappers around popular embedding providers, making it easy to use them in your apps. You can set an embedding function when you create a Chroma collection, to be automatically used when adding and querying data, or you can call them directly yourself.

	Python	Typescript
Cloudflare Workers AI	✓	✓
Cohere	✓	✓
Google Generative AI	✓	✓
Hugging Face	✓	-
Hugging Face Embedding Server	✓	✓
Jina AI	✓	✓
Mistral	✓	✓
Morph	✓	✓
OpenAI	✓	✓
Sentence Transformers	✓	✓
Together AI	✓	✓

For TypeScript users, Chroma provides packages for a number of embedding model providers. The Chromadb python package ships with all embedding functions included.

Provider	Embedding Function Package
All (installs all packages)	@chroma-core/all
Cloudflare Workers AI	@chroma-core/cloudflare-worker-ai
Cohere	@chroma-core/cohere
Google Gemini	@chroma-core/google-gemini

Provider	Embedding Function Package
Hugging Face Server	@chroma-core/huggingface-server
Jina	@chroma-core/jina
Mistral	@chroma-core/mistral
Morph	@chroma-core/morph
Ollama	@chroma-core/ollama
OpenAI	@chroma-core/openai
Perplexity	@chroma-core/perplexity
Qwen (via Chroma Cloud)	@chroma-core/chroma-cloud-qwen
Sentence Transformers	@chroma-core/sentence-transformer
Together AI	@chroma-core/together-ai
Voyage AI	@chroma-core/voyageai

We welcome contributions! If you create an embedding function that you think would be useful to others, please consider submitting a pull request.

Multimodal Embeddings

Source: https://docs.trychroma.com/docs/embeddings/multimodal

> **Warning:** Multimodal support is currently available only in Python. Javascript/Typescript support coming soon!

You can create multimodal Chroma collections; these are collections which can store, and can be queried by, multiple modalities of data.

Try it out in Colab

Multi-modal Embedding Functions

Chroma supports multi-modal embedding functions, which can be used to embed data from multiple modalities into a single embedding space.

Chroma ships with the OpenCLIP embedding function built in, which supports both text and images.

```
from chromadb.utils.embedding_functions import OpenCLIPEmbeddingFunction
embedding_function = OpenCLIPEmbeddingFunction()
```

Adding Multimodal Data and Data Loaders

You can add embedded data of modalities different from text directly to Chroma. For now images are supported:

```
collection.add(
    ids=['id1', 'id2', 'id3'],
    images=[[1.0, 1.1, 2.1, ...], ...] # A list of numpy arrays representing images
)
```

Unlike with text documents, which are stored in Chroma, we will not store your original images, or data of other modalities. Instead, for each of your multimodal records you can specify a URI where the original format is stored, and a **data loader**. For each URI you add, Chroma will use the data loader to retrieve the original data, embed it, and store the embedding.

For example, Chroma ships with a data loader, `ImageLoader`, for loading images from a local filesystem. We can create a collection set up with the `ImageLoader`:

```
from chromadb.utils.data_loaders import ImageLoader
from chromadb.utils.embedding_functions import OpenCLIPEmbeddingFunction

client = chromadb.Client()

data_loader = ImageLoader()
embedding_function = OpenCLIPEmbeddingFunction()

collection = client.create_collection(
    name='multimodal_collection',
    embedding_function=embedding_function,
    data_loader=data_loader
)
```

Now, we can use the `.add` method to add records to this collection. The collection's data loader will grab the images using the URIs, embed them using the `OpenCLIPEmbeddingFunction`, and store the embeddings in Chroma.

```
collection.add(
    ids=["id1", "id2"],
    uris=["path/to/file/1", "path/to/file/2"]
)
```

If the embedding function you use is multi-modal (like `OpenCLIPEmbeddingFunction`), you can also add text to the same collection:

```
collection.add(
    ids=["id3", "id4"],
    documents=["This is a document", "This is another document"]
)
```

Querying

You can query a multi-modal collection with any of the modalities that it supports. For example, you can query with images:

```
results = collection.query(
    query_images=[...] # A list of numpy arrays representing images
)
```

Or with text:

```
results = collection.query(
    query_texts=["This is a query document", "This is another query document"]
)
```

If a data loader is set for the collection, you can also query with URIs which reference data stored elsewhere of the supported modalities:

```
results = collection.query(
    query_uris=[...] # A list of strings representing URIs to data
)
```

Additionally, if a data loader is set for the collection, and URIs are available, you can include the data in the results:

```
results = collection.query(
    query_images=[...], # # list of numpy arrays representing images
    include=['data']
)
```

This will automatically call the data loader for any available URIs, and include the data in the results. `uris` are also available as an `include` field.

Updating

You can update a multi-modal collection by specifying the data modality, in the same way as `add`. For now, images are supported:

```
collection.update(
    ids=['id1', 'id2', 'id3'],
    images=[...] # A list of numpy arrays representing images
)
```

Note that a given entry with a specific ID can only have one associated modality at a time. Updates will over-write the existing modality, so for example, an entry which originally has corresponding text and updated with an image, will no longer have that text after an update with images.

Ollama

Source: https://docs.trychroma.com/integrations/embedding-models/ollama

Chroma provides a convenient wrapper around Ollama's embeddings API. You can use the `OllamaEmbeddingFunction` embedding function to generate embeddings for your documents with a model of your choice.

Python

```python
from chromadb.utils.embedding_functions.ollama_embedding_function import (
    OllamaEmbeddingFunction,
)

ollama_ef = OllamaEmbeddingFunction(
    url="http://localhost:11434",
    model_name="llama2",
)

embeddings = ollama_ef(["This is my first text to embed",
                        "This is my second document"])
```

Nomic

Source: https://docs.trychroma.com/integrations/embedding-models/nomic

Chroma provides a convenient wrapper around Nomic's embedding API. This embedding function runs remotely on Nomic's servers, and requires an API key. You can get an API key by signing up for an account at Nomic.

Python

This embedding function relies on the `nomic` python package, which you can install with `pip install nomic`.

```
from chromadb.utils.embedding_functions import NomicEmbeddingFunction


os.environ["NOMIC_API_KEY"] = "YOUR_API_KEY"
nomic_ef = NomicEmbeddingFunction(
    model="nomic-embed-text-v1",
    task_type="search_document",
    query_config={"task_type": "search_query"}
)

texts = ["Hello, world!", "How are you?"]
embeddings = nomic_ef(texts)
```

You must pass in a `model` argument and `task_type` argument. The `task_type` can be one of: - `search_document`: Used to encode large documents in retrieval tasks at indexing time - `search_query`: Used to encode user queries or questions in retrieval tasks - `classification`: Used to encode text for text classification tasks - `clustering`: Used for clustering or reranking tasks

The `query_config` parameter allows you to specify a different task type for queries, which is useful when you want to use `search_document` for documents and `search_query` for queries.

> **Note:** Visit Nomic documentation for more information on available models and task types.

Langchain

Source: https://docs.trychroma.com/integrations/frameworks/langchain

Langchain - Python

- LangChain + Chroma on the LangChain blog
- Harrison's `chroma-langchain` demo repo
- question answering over documents - (Replit version)
- to use Chroma as a persistent database
- Tutorials
- Chroma and LangChain tutorial - The demo showcases how to pull data from the English Wikipedia using their API. The project also demonstrates how to vectorize data in chunks and get embeddings using OpenAI embeddings model.
- Create a Voice-based ChatGPT Clone That Can Search on the Internet and local files
- LangChain's Chroma Documentation

Langchain - JS

- LangChainJS Chroma Documentation

Intro to Retrieval

Source: https://docs.trychroma.com/guides/build/intro-to-retrieval

Large language models like GPT-5, Claude, or Gemini can write code, answer questions, generate content, and solve complex problems with remarkable sophistication. You can build chatbots, content generators, coding assistants, and analysis tools by crafting good prompts and calling AI APIs.

However, as you start to build more sophisticated applications, especially those that require knowledge not available to an LLM, you'll quickly run into some fundamental limitations.

The Knowledge Problem

Imagine you're building an AI assistant for a company's customer support team. You want the AI to answer questions about your products, policies, and procedures.

Given a simple prompt

> What is your return policy for tooth brushes?

The AI is likely to return a generic answer, because it doesn't know about your company's specific policies:

> I'm not a retailer or company that sells toothbrushes - I'm Claude, an AI assistant created by Anthropic. I don't have products or a return policy. If you're looking for information about a toothbrush return policy, you'd need to check with the specific store or company where you purchased or are planning to purchase the toothbrush. Most retailers have their return policies listed on their websites or you can contact their customer service directly.

We can get around this problem, by providing useful information or context for the AI. This often called "context engineering":

> What is your return policy for tooth brushes? Use the following information for context: Tooth brushes can be returned up to 360 days after purchase if unopened. Tooth brushes are unable to be returned if they are used. Tooth brushes can be returned if they suffer manufacturing defects discovered after opening.

Now the AI has the information it needs to answer the user question correctly:

> Based on the information provided, our return policy for toothbrushes allows returns up to 360 days after purchase, as long as the toothbrush remains unopened and in its original packaging. This gives you nearly a full year to return an unopened toothbrush if needed. Once a toothbrush has been opened, it wouid not be eligible for return under this policy unless it has manufacturing defects.

But what happens if we want our AI chatbot to be able to answer questions about shipping, product specifications, troubleshooting, or any other topic? We will have to include our entire knowledge base in our context, which leads to several technical problems.

Token Limits: AI models have maximum input lengths. Even the largest models might not be able to fit an entire company's documentation in a single prompt.

Cost: AI APIs typically charge per token. Including thousands of words of context in every request becomes expensive quickly.

Relevance: When you include too much information, the AI can get confused or focus on irrelevant details instead of what actually matters for answering the user's question.

Freshness: Information changes constantly. Product specs update, policies change, new documentation gets written. Keeping everything in prompts means manually updating every prompt whenever anything changes.

Hallucinations: Without the correct information or focus for answering a user's question, LLMs may produce a wrong answer with an authoritative voice. For most business applications, where accuracy matters, hallucination is a critical problem.

Enter Retrieval

Retrieval solves these fundamental challenges by creating a bridge between AI models and your actual data. Instead of trying to cram everything into prompts, a retrieval system **stores your information** in a searchable format. This allows you to search your knowledge base using natural language, so you can find relevant information to answer the user's question, by providing the retrieval system with the user's question itself. This way, you can build context for the model in a strategic manner.

When a retrieval system returns the results from your knowledge base relevant to the user's question, you can use them to provide context for the AI model to help it generate an accurate response.

Here's how a typical retrieval pipeline is built:

1. **Converting information into searchable formats** - this is done by using **embedding models**. They create mathematical representations of your data, called "embeddings", that capture the semantic meaning of text, not just keywords.
2. **Storing these representations** in a retrieval system, optimized for quickly finding similar embeddings for an input query.
3. **Processing user queries** into embeddings, so they can be used as inputs to your retrieval system.
4. **Query and retrieve** results from the database.
5. **Combining the retrieved results** with the original user query to serve to an AI model.

Chroma is a powerful retrieval system that handles most of this process out-of-the-box. It also allows you to customize these steps to get the best performance in your AI application. Let's see it in action for our customer support example.

Step 1: Embed our Knowledge Base and Store it in a Chroma Collection

Python

Install Chroma:

pip

```
pip install chromadb
```

poetry

```
poetry add chromadb
```

uv

```
uv pip install chromadb
```

Chroma embeds and stores information in a single operation.

```
client = chromadb.Client()
customer_support_collection = client.create_collection(
    name="customer support"
)

customer_support_collection.add(
   ids=["1", "2", "3"],
   documents=[
      "Toothbrushes can be returned up to 360 days after purchase if unopened.",
      "Shipping is free of charge for all orders.",
      "Shipping normally takes 2-3 business days"
   ]
)
```

Step 2: Process the User's Query

Similarly, Chroma handles the embedding of queries for you out-of-the-box.

Python

```
user_query = "What is your return policy for tooth brushes?"

context = customer_support_collection.query(
    queryTexts=[user_query],
    n_results=1
)['documents'][0]

print(context) # Toothbrushes can be returned up to 360 days after purchase if unopened.
```

Step 3: Generate the AI Response

With the result from Chroma, we can build the correct context for an AI model.

OpenAI

Python

```
openai.api_key = os.getenv("OPENAI_API_KEY")

prompt = f"{user_query}. Use this as context for answering: {context}"

response = openai.ChatCompletion.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": "You are a helpful assistant"},
        {"role": "user", "content": prompt}
    ]
)
```

Anthropic

Python

```
client = anthropic.Anthropic(
    api_key=os.getenv("ANTHROPIC_API_KEY")
)

prompt = f"{user_query}. Use this as context for answering: {context}"

response = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    messages=[
        {"role": "user", "content": prompt}
    ]
)
```

There's a lot left to consider, but the core building blocks are here. Some next steps to consider:

- **Embedding Model** There are many embedding models on the market, some optimized for code, others for english and others still for various languages. Embedding model selection plays a big role in retrieval accuracy.
- **Chunking** Chunking strategies are very unique to the data. Deciding how large or small to make chunks is critical to the performance of the system.
- **n_results** varying the number of results balances token usage with correctness. The more results, the likely the better answer from the LLM but at the expense of more token usage.

Chunking

Source: https://docs.trychroma.com/guides/build/chunking

Retrieval-Augmented Generation (RAG) lets us ground large language models in our own data. The core idea is simple: we store our data in a Chroma collection. Then, before issuing a request to an LLM, we find the relevant parts of data in the collection, and include them in the prompt so the LLM can answer based on real information rather than its training data alone.

But here's the problem: we can't just throw entire documents at the model. For example, a single PDF from our data might contain 50 pages. A codebase might span thousands of files. Even a modest knowledge base can exceed what fits in a context window - and even when documents do fit, including entire files is wasteful. If someone asks "What's the default timeout?", we don't want to retrieve a 20-page configuration guide; we want the specific paragraph that answers the question.

Beyond the context concerns, we also need to be mindful of how we embed and store data. All embedding models have their own token limits. If we try to embed a document exceeding this limit, the resulting embedding will not represent the parts of the document beyond the model's limit. Additionally, Chroma limits each record document size to 16KB.

This is why RAG systems work with **chunks** - smaller pieces of documents that can be independently retrieved based on relevance to a query.

A common **ingestion pipeline** works as follows: we split data into chunks, collect metadata fields we can attach to each chunk, and insert the resulting records into our Chroma collection. Chroma will automatically embed the chunks using the collection's embedding function.

Choosing Chunking Boundaries

Chunking forces a trade-off: chunks need to be small enough to match specific queries, but large enough to be self-contained and meaningful.

Consider building a chatbot over technical documentation, where we decide to chunk text by sentences. The following paragraph

> The connection timeout controls how long the client waits when establishing a connection to the server. The default value is 30 seconds. For high-latency networks, consider increasing this to 60 seconds. Note that this is different from the read timeout, which controls how long the client waits for data after the connection is established.

Will produce these chunks:

- **Chunk 1**: "The connection timeout controls how long the client waits when establishing a connection to the server."
- **Chunk 2**: "The default value is 30 seconds."
- **Chunk 3**: "For high-latency networks, consider increasing this to 60 seconds."
- **Chunk 4**: "Note that this is different from the read timeout, which controls how long the client waits for data after the connection is established."

Now a user asks:

How long is the connection timeout?

Chunk 2 contains "The default value is 30 seconds"-but it never mentions "connection timeout." That phrase only appears in Chunk 1. When we issue this query to the collection, we have no guarantee that both chunks will be retrieved so an LLM can compile the correct answer.

A better approach keeps full paragraphs together, so the answer and its context share the same embedding and get retrieved as a unit. The right boundaries depend on what we're chunking. A novel has different natural units than an API reference. Code has different logical boundaries than an email thread.

Poor chunking creates a chain of problems through your pipeline:

1. Retrieval returns partial matches. In the example above, searching for "default connection timeout" might rank Chunk 1 highest (it mentions "connection timeout") even though Chunk 2 has the actual answer. Your relevance scores look reasonable, but the retrieved content doesn't actually answer the question.
2. You compensate by increasing top-k. When individual chunks don't contain complete information, you retrieve 10 or 20 results instead of 3 or 4. This increases token costs, and dilutes the prompt with marginally relevant text-hurting the LLM's ability to focus on what matters.
3. The LLM produces degraded answers. The model can only synthesize what you provide. Fragmentary context leads to hedged answers ("The default value appears to be 30 seconds, but I'm not certain what parameter this refers to..."), hallucinated details, or outright errors.

Chunking Strategies

Recursive splitting - Try to split at the largest structural unit first (e.g., double newlines for paragraphs), but if a resulting chunk exceeds your size limit (token and/or document limit), recursively split it using smaller units (single newlines, then sentences, then words). This balances structure-awareness with size constraints. LangChain's `RecursiveCharacterTextSplitter` is a common implementation.

Split with Overlap - Use a chunking strategy (like recursive splitting), but include an overlap between chunks. For example, if splitting a PDF by paragraphs, Chunk-1 contains the first paragraph and the first sentence of the second paragraph. Chunk-2 contains the second paragraph and the last sentence of the first paragraph. The overlap creates redundancy that helps preserve context across boundaries. The downside: you're storing and embedding duplicate content.

Structure-aware splitting - Parse the document's explicit structure: Markdown headers, HTML DOM, or code ASTs. Split at structural boundaries and optionally include hierarchical context in the chunk's content itself. For example, when splitting the code for a class by instance methods, include at the top of each chunk a code comment mentioning the encompassing class, file name, etc.

Semantic splitting - Embed sentences or paragraphs, compute similarity between adjacent segments, and place chunk boundaries where similarity drops (indicating a topic shift). This process can also be driven by an LLM alternatively. This method is more computationally expensive but can produce more coherent chunks when documents lack clear structural markers.

> **Note:** Learn more about different strategies in our chunking research report

Chunking Text

For most text documents, recursive chunking with some chunk overlap is a good starting point. LangChain's `RecursiveCharacterTextSplitter` is an example implementation for this strategy. It tries to split at natural boundaries (paragraphs first, then sentences, then words) while respecting size limits and adding overlap to preserve context across boundaries.

Python

```python
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50,
    separators=["\n\n", "\n", ". ", " "]
)

chunks = splitter.split_text(document)
```

When chunking Markdown files, we can take advantage of their structure. For example, we can split by headers - try to split by `h2` headers, and recursively try inner headers.

We can also contextualize each chunk by specifying its place in the document's structure. For example, if end up with a chunk that is under an `h3` header, we can append at the top the path from the document's `h1` to this chunk.

LangChain's `MarkdownHeaderTextSplitter` splits by section and captures the header hierarchy as metadata.

Python

```python
from langchain.text_splitter import MarkdownHeaderTextSplitter

splitter = MarkdownHeaderTextSplitter(
    headers_to_split_on=[("#", "h1"), ("##", "h2"), ("###", "h3")]
)
chunks = splitter.split_text(markdown_doc)
```

Each chunk includes the path to it from the document's `h1` header:

```
{
  "h1": "Config",
  "h2": "Timeouts"
}
```

We can leverage it to add this context for each chunk:

Python

```python
def contextualize(chunk) -> str:
    headers = [chunk.metadata.get(f"h{i}") for i in range(1, 4)]
    path = " > ".join(h for h in headers if h)
    return f"[{path}]\n\n{chunk.page_content}" if path else chunk.page_content
```

Chunking Code

When chunking text-based files, our split boundaries are often obvious - paragraphs, sentences, Markdown headers, etc. Code is trickier - there's no single obvious unit. Functions? Classes? Files? Instance methods can be too granular, files too large, and the right choice often depends on the codebase and the types of queries you want to answer.

Using the same idea that chunks should be self-contained units of our data, we will choose classes and functions as our chunking boundaries, and treat them as atomic units of code that should not be broken down further.

This way, if a query like "how is auth handled" is submitted, we can get back a chunk containing a relevant function. If that chunk contains references to other classes or functions, we can subsequently retrieve the chunks where they are represented (via regex search for example).

A great tool that gives us the ability to parse a file of code into these units is `tree-sitter`. It is a fast parsing library that can build an abstract syntax tree, or an AST, for an input source code.

For example, if we parse this code snippet with tree sitter:

```
class MyClass:
    def say_hello(self, name: str) -> None:
        print(f"Hello {name}")
```

We will get a tree with a `class_definition` node, which encompasses the entire class. It will have as a child a `method_definition` node, covering the `say_hello` method, and so on.

Each node represents a construct of the language we work with, which is exactly what we want to have in our collection.

A Small Example

Let's examine a small example of using `tree-sitter` to parse Python files. To being, we'll set up `tree-sitter` and a parser for Python files:

Python

```
pip install tree-sitter tree-sitter-python
```

Python

```
from tree_sitter import Language, Parser


# Use Python grammar
python_language = Language(tspython.language())

# Set up the parser
parser = Parser(python_language)
```

Using the parser, we can process the code snippet from our small example:

Python

```
source_code = b"""
class MyClass:
    def say_hello(self, name: str) -> None:
        print(f"Hello {name}")
"""

tree = parser.parse(source_code)
root = tree.root_node
```

The root node encompasses the entire source code. Its first child is the `class_definition` node, spanning lines 1-3. If we explore further down the tree, we will find the `function_definition` node, which spans lines 2-3.

Python

```python
print(root.children[0])
# <Node type=class_definition, start_point=(1, 0), end_point=(3, 30)>

print(root.children[0].children[3].children[0])
# <Node type=function_definition, start_point=(2, 4), end_point=(3, 30)>
```

Recursively Exploring an AST

We can write a function, that given source code, parses it using the `tree-sitter` parser, and recursively explores the tree to find the nodes we want represented in our chunks. Recall that we wanted to treat our "target" node as atomic units. So we will stop the recursion when we find such nodes.

We can also use the nodes' `start_byte` and `end_byte` fields to get back the code each node represents. `tree-sitter` can also give us the line numbers each node spans, which we can save in chunks' metadata:

Python

```python
from uuid import uuid4

def parse_code(file_path: str) -> list[Chunk]:
    with open(file_path, "rb") as f:
        source_code = f.read()

    tree = parser.parse(source_code)
    root = tree.root_node

    target_types = ['function_definition', 'class_definition']

    def collect_nodes(node: Node) -> list[Node]:
        result: list[Node] = []

        if node.type in target_types:
            result.append(node)
        else:
            for child in node.children:
                result.extend(collect_nodes(child))

        return result

    nodes = collect_nodes(root)
    chunks = []

    for node in nodes:
        name_node = node.child_by_field_name("name")
        symbol = source_code[name_node.start_byte:name_node.end_byte].decode()
        chunk = Chunk(
            id=str(uuid4()),
            content=source_code[node.start_byte : node.end_byte].decode("utf-8"),
            start_line=node.start_point[0],
            end_line=node.end_point[0],
            path=file_path,
        )
        chunks.append(chunk)

    return chunks
```

If the chunks this method produces are still too large, we can default to splitting them by line spans. If we ever need to reconstruct them, we can use the line-number metadata fields.

Evaluation

To evaluate your chunking strategy, test it against real queries and measure how well the right chunks surface. The goal is retrieval quality: when we issue a query to Chroma, do the top results contain the information needed to answer it?

Create a set of test queries with ground truth: each query maps to the chunk(s) that should be retrieved for it:

Python

```
test_queries = [
    {
        "query": "What's the default connection timeout?",
        "expected_chunks": ["chunk-3"],
    },
    {
        "query": "How do I authenticate with OAuth?",
        "expected_chunks": ["chunk-1", "chunk-2"],
    },
    # ...
]
```

The key metrics you will measure are: * **Recall@k**: Of your test queries, what percentage have the correct chunk in the top `k` results?

Python

```
def recall_at_k(results: list[str], expected: list[str], k: int) -> float:
    top_k = set(results[:k])
    return len(top_k & set(expected)) / len(expected)
```

- **Mean Reciprocal Rank (MRR)** - Where does the first correct chunk appear? (Higher is better)

Python

```
def mrr(results: list[str], expected: list[str]) -> float:
    for i, chunk_id in enumerate(results):
        if chunk_id in expected:
            return 1 / (i + 1)
    return 0
```

Then test your queries against the chunks in your collection:

Python

```
k = 10

results = collection.query(
    query_texts=[test_case["query"] for test_case in test_queries],
    n_results=k
)

metrics = [
    {
        "recall": recall_at_k(chunk_ids, test_queries[i]["expected_chunks"], k),
        "mrr": mrr(chunk_ids, test_queries[i]["expected_chunks"])
    }
    for i, chunk_ids in enumerate(results["ids"])
]
```

If you see: * Low recall (the correct chunks are not in the top-k results) - try smaller chunks, with more overlap between them. * Correct chunks rank low - add context to the chunks themselves and leverage metadata filtering * Duplicate results - decrease chunk overlap * Irrelevant matches - try larger chunks, structure-aware chunking, or semantic-aware chunking.

Look at Your Data

Source: https://docs.trychroma.com/guides/build/look-at-your-data

Before building our RAG pipelines and inserting data into Chroma collections, it is worth asking ourselves the following questions:

- What types of searches do we want to support? (semantic, regex, keyword, etc.)
- What embedding models should we use for semantic and keyword searches?
- Should chunks live in one Chroma collection, or should we use different collections for different chunk types?
- What are the meaningful units of data we want to store as records in our Chroma collections?
- What metadata fields can we leverage when querying?

The structure of our collections, the granularity of our chunks, and the metadata we capture - all directly impact retrieval quality-and by extension, the quality of the LLM's responses in our AI application.

Search Modalities

Chroma supports various search techniques that are useful for different use cases.

Dense search (semantic) uses embeddings to find records that are semantically similar to a query. It excels at matching meaning and intent - a query like "how do I return a product" can surface relevant chunks even if they never use the word "return." The weakness? Dense search can struggle with exact terms: product SKUs, part numbers, legal case citations, or domain-specific jargon that didn't appear often in the embedding model's training data.

All Chroma collections enable semantic search by default. You can specify the embedding function your collection will use to embed your data when creating a collection:

Python

```python
from chromadb.utils.embedding_functions import OpenAIEmbeddingFunction

client = chromadb.CloudClient()

collection = client.create_collection(
    name="my-collection",
    embedding_function=OpenAIEmbeddingFunction(
        api_key="YOUR_OPENAI_API_KEY",
        model="text-embedding-3-small"
    )
)
```

Lexical search (keyword) matches on exact tokens. It shines when you need precision: finding a specific product ID like `SKU-4892-X`, a drug name like `omeprazole`, a legal citation like `Smith v. Jones (2019)`, or a model number in a technical manual. Dense search might miss these entirely or return semantically related but wrong results. The tradeoff is that lexical search can't bridge synonyms or paraphrases - searching "cancel" won't find chunks that only mention "terminate."

To enable lexical search on your collection, you can enable a sparse vector index on your collection's schema with a sparse embedding function:

Python

```python
from chromadb import Schema, SparseVectorIndexConfig, K
from chromadb.utils.embedding_functions import ChromaCloudSpladeEmbeddingFunction

client = chromadb.CloudClient()

schema = Schema()

schema.create_index(
    config=SparseVectorIndexConfig(
        source_key=K.DOCUMENT,
        embedding_function=ChromaCloudSpladeEmbeddingFunction()
    ),
    key="sparse_embedding"
)

collection = client.create_collection(
    name="my-collection",
    schema=schema
)
```

Hybrid search combines both: run dense and lexical searches in parallel, then merge the results. This gives you semantic understanding and precise term matching. For many retrieval tasks - especially over technical or specialized content - hybrid outperforms either approach alone.

Chroma's Search API allows you to define how you want to combine dense and sparse (lexical) results. For example, using RRF:

Python

```python
from chromadb import Search, K, Knn, Rrf

# Dense semantic embeddings
dense_rank = Knn(
    query="machine learning research",  # Text query for dense embeddings
    key="#embedding",          # Default embedding field
    return_rank=True,
    limit=200                  # Consider top 200 candidates
)

# Sparse keyword embeddings
sparse_rank = Knn(
    query="machine learning research",  # Text query for sparse embeddings
    key="sparse_embedding",    # Metadata field for sparse vectors
    return_rank=True,
    limit=200
)

# Combine with RRF
hybrid_rank = Rrf(
    ranks=[dense_rank, sparse_rank],
    weights=[0.7, 0.3],        # 70% semantic, 30% keyword
    k=60
)

# Use in search
search = (Search()
    .where(K("status") == "published")  # Optional filtering
    .rank(hybrid_rank)
    .limit(20)
    .select(K.DOCUMENT, K.SCORE, "title")
)

results = collection.search(search)
```

Chroma also supports **text filtering** on top of your searches via the `where_document` parameter. You can filter results to only include chunks that contain an exact string or match a regex pattern.

This is useful for enforcing structural constraints-like ensuring results contain a specific identifier-or for pattern matching on things like email addresses, dates, or phone numbers.

Embedding Models

Dense embedding models map text to vectors where semantic similarity is captured by vector distance.

Chroma has first-class support for many embedding models. The tradeoffs include cost (API-based vs. local), latency, embedding dimensions (which affect storage and search speed), and quality on your specific domain. General-purpose models work well for most text, but specialized models trained on code, legal documents, or medical text can outperform them on domain-specific tasks. Larger models typically produce better embeddings but cost more and run slower-so the right choice depends on your quality requirements and constraints. * If you're building a customer support bot over general documentation, a model like `text-embedding-3-small` offers a good balance of quality and cost. * For a codebase search tool, code-specific models will better capture the semantics of function names, syntax, and programming patterns. Chroma works with code-specific models from OpenAI, Cohere, Mistral, Morph, and more. * If you need to run entirely locally for privacy or cost reasons, smaller open-source models like `all-MiniLM-L6-v2` are a practical choice, though with some quality tradeoff.

Sparse embedding models power lexical search. For example, BM25 counts the frequency of tokens in a document and produces a vector representing the counts for each token. When we issue a lexical search query, we will get back the documents whose sparse vectors have a higher count for the tokens in our query.

SPLADE is a learned alternative that expands terms-so a document about "dogs" might also get weight on "puppy" and "canine," helping bridge the synonym gap that pure lexical search misses. * If your data contains lots of exact identifiers that must match precisely - SKUs, legal citations, chemical formulas - BM25 is straightforward and effective. * If you want lexical search that's more forgiving of vocabulary mismatches, SPLADE can help.

Collections in your Chroma Database

A Chroma collection indexes records using a specific embedding model and configuration. Whether your records live in one Chroma collection or many depends on your application's access patterns and data types.

Use a single collection when: * You are using the same embedding model for all of your data. * You want to search across everything at once. * You can distinguish between records using metadata filtering.

Use multiple collections when: * You have different types of data, requiring different embedding models. For example, you have text data and images, which are embedded using different models. * You have multi-tenant requirements. In this case, establishing a collection per user or organization helps you avoid filtering overhead at query time.

Chunking Data

Chunking is the process of breaking source data into smaller, meaningful units ("chunks") that are embedded and stored as individual records in a Chroma collection. Because embedding models operate on limited context windows and produce a single vector per input, storing entire documents as one record often blurs multiple ideas together and reduces retrieval quality.

Chunking allows Chroma to index information at the level users actually search for-paragraphs, sections, functions, or messages-improving both recall and precision. Well-chosen chunks ensure that retrieved results are specific, semantically coherent, and useful on their own, while still allowing larger context to be reconstructed through metadata when needed.

> **Note:** To learn more about chunking best practices, see our Chunking Guide

Chroma is flexible enough to support nearly any chunking strategy so long as each chunk fits in 16kB. This is also the best way to work with large documents, regardless of performance concerns.

When adding chunks to your collection, we recommend using batch operations. Batching increases the number of items sent per operation, acting as a throughput multiplier. Going from one vector to two will generally double the number of vectors per second with diminishing returns as the batch size increases. Chroma Cloud allows ingesting up to 300 vectors per batch.

Python

```python
# Instead of
for chunk in chunks:
    collection.add(
        ids=[chunk.id],
        documents=[chunk.document],
        metadatas=[chunk.metadata]
    )

# Use batching
BATCH_SIZE = 300
for i in range(0, len(chunks), BATCH_SIZE):
    batch = chunks[i:i + BATCH_SIZE]
    collection.add(
        ids=[chunk.id for chunk in batch],
        documents=[chunk.document for chunk in batch],
        metadatas=[chunk.metadata for chunk in batch]
    )
```

Finally, issuing concurrent requests to the same collection will allow for even more throughput. Internally, requests are batched to give better performance than would be seen issuing requests individually. This batching happens automatically and to greater numbers than the 300 vectors per batch permitted by default. Every Chroma Cloud user can issue up to 10 concurrent requests.

Metadata

Metadata lets you attach structured information to each chunk, which serves two purposes: filtering at query time and providing context to the LLM.

For filtering, metadata lets you narrow searches without relying on semantic similarity. You might filter by source type (only search FAQs, not legal disclaimers), by date (only recent documents), by author or department, or by access permissions (only return chunks the user is allowed to see). This is often more reliable than hoping the embedding captures these distinctions.

Metadata is also returned with search results, which means you can pass it to the LLM alongside the chunk text. Knowing that a chunk came from "Q3 2024 Financial Report, page 12" or "authored by the legal team" helps the LLM interpret the content and cite sources accurately.

When designing your schema, think about what filters you'll need at query time and what context would help the LLM make sense of each chunk.

Building with AI

Source: https://docs.trychroma.com/guides/build/building-with-ai

AI is a new type of programming primitive. Large language models (LLMs) let us write software which can process **unstructured** information in a **common sense** way.

Consider the task of writing a program to extract a list of people's names from the following paragraph:

> Now the other princes of the Achaeans slept soundly the whole night through, but Agamemnon son of Atreus was troubled, so that he could get no rest. As when fair Hera's lord flashes his lightning in token of great rain or hail or snow when the snow-flakes whiten the ground, or again as a sign that he will open the wide jaws of hungry war, even so did Agamemnon heave many a heavy sigh, for his soul trembled within him. When he looked upon the plain of Troy he marveled at the many watchfires burning in front of Ilion... - The Iliad, Scroll 10

Extracting names is easy for humans, but is very difficult using only traditional programming. Writing a general program to extract names from any paragraph is harder still.

However, with an LLM the task becomes almost trivial. We can simply provide the following input to an LLM:

> List the names of people in the following paragraph, separated by commas: Now the other princes of the Achaeans slept soundly the whole night through, but Agamemnon son of Atreus was troubled, so that he could get no rest. As when fair Hera's lord flashes his lightning in token of great rain or hail or snow when the snow-flakes whiten the ground, or again as a sign that he will open the wide jaws of hungry war, even so did Agamemnon heave many a heavy sigh, for his soul trembled within him. When he looked upon the plain of Troy he marveled at the many watchfires burning in front of Ilion... - The Iliad, Scroll 10

The output would correctly be:

> Agamemnon, Atreus, Hera

Integrating LLMs into software applications is as simple as calling an API. While the specifics of the API may vary between LLMs, most have converged on some common patterns:

- Calls to the API typically consist of parameters including a `model` identifier, and a list of `messages`.
- Each `message` has a `role` and `content`.
- The `system` role can be thought of as the *instructions* to the model.
- The `user` role can be thought of as the *data* to process.

For example, we can use AI to write a general purpose function that extracts names from input text.

OpenAI

Python

```
openai.api_key = os.getenv("OPENAI_API_KEY")
```

```
def extract_names(text: str) -> list[str]:
    system_prompt = "You are a name extractor. The user will give you text, and you must
return a JSON array of names mentioned in the text. Do not include any explanation or
formatting."

    response = openai.ChatCompletion.create(
        model="gpt-4o",
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": text}
        ]
    )

    response = response.choices[0].message["content"]
    return json.loads(response)
```

Anthropic

Python

```
client = anthropic.Anthropic(
    api_key=os.getenv("ANTHROPIC_API_KEY")
)

def extract_names(text: str) -> list[str]:
    system_prompt = "You are a name extractor. The user will give you text, and you must
return a JSON array of names mentioned in the text. Do not include any explanation or
formatting."

    response = client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=1000,
        system=system_prompt,
        messages=[
            {"role": "user", "content": text}
        ]
    )

    response_text = response.content[0].text
    return json.loads(response_text)
```

Building with AI allows new type of work to be done by software. LLMs are capable of understanding abstract ideas and take action. Given access to retrieval systems and tools, LLMs can operate on tasks autonomously in ways that wasn't possible with classical software.

Client

Source: https://docs.trychroma.com/reference/python/client

Clients

EphemeralClient

Create an in-memory client for local use.

This client stores all data in memory and does not persist to disk. It is intended for testing and development.

- `settings` (Optional[Settings]): Optional settings to override defaults.
- `tenant` (str): Tenant name to use for requests. Defaults to the default tenant.
- `database` (str): Database name to use for requests. Defaults to the default database.

PersistentClient

Create a persistent client that stores data on disk.

This client is intended for local development and testing. For production, prefer a server-backed Chroma instance.

- `path` (Union[str, Path]): Directory to store persisted data.
- `settings` (Optional[Settings]): Optional settings to override defaults.
- `tenant` (str): Tenant name to use for requests.
- `database` (str): Database name to use for requests.

HttpClient

Create a client that connects to a Chroma server.

- `host` (str): Hostname of the Chroma server.
- `port` (int): HTTP port of the Chroma server.
- `ssl` (bool): Whether to enable SSL for the connection.
- `headers` (Optional[Dict[str, str]]): Optional headers to send with each request.
- `settings` (Optional[Settings]): Optional settings to override defaults.
- `tenant` (str): Tenant name to use for requests.
- `database` (str): Database name to use for requests.

AsyncHttpClient

Create an async client that connects to a Chroma HTTP server.

This supports multiple clients connecting to the same server and is the recommended production configuration.

- **`host`** (str): Hostname of the Chroma server.
- **`port`** (int): HTTP port of the Chroma server.
- **`ssl`** (bool): Whether to enable SSL for the connection.
- **`headers`** (Optional[Dict[str, str]]): Optional headers to send with each request.
- **`settings`** (Optional[Settings]): Optional settings to override defaults.
- **`tenant`** (str): Tenant name to use for requests.
- **`database`** (str): Database name to use for requests.

CloudClient

Create a client for Chroma Cloud.

If not provided, `tenant`, `database`, and `api_key` will be inferred from the environment variables `CHROMA_TENANT`, `CHROMA_DATABASE`, and `CHROMA_API_KEY`.

- **`tenant`** (Optional[str]): Tenant name to use, or None to infer from credentials.
- **`database`** (Optional[str]): Database name to use, or None to infer from credentials.
- **`api_key`** (Optional[str]): API key for Chroma Cloud.
- **`settings`** (Optional[Settings]): Optional settings to override defaults.

AdminClient

Create an admin client for tenant and database management.

Client Methods

heartbeat

Get the current time in nanoseconds since epoch.

Used to check if the server is alive.

Returns: The current time in nanoseconds since epoch

list_collections

List all collections.

- **`limit`** (Optional[int]): The maximum number of entries to return. Defaults to None.
- **`offset`** (Optional[int]): The number of entries to skip before returning. Defaults to None.

Returns: A list of collections

count_collections

Count the number of collections.

Returns: The number of collections.

create_collection

Create a new collection with the given name and metadata.

- `name` (str, required): The name of the collection to create.
- `metadata` (Optional[Dict[str, Any]]): Optional metadata to associate with the collection.
- `embedding_function` (Optional[EmbeddingFunction[Optional[Embeddings]]]): Optional function to use to embed documents. Uses the default embedding function if not provided.
- `data_loader` (Optional[DataLoader[Optional[Embeddings]]]): Optional function to use to load records (documents, images, etc.)
- `get_or_create` (bool): If True, return the existing collection if it exists.

Returns: The newly created collection.

Raises:

- ValueError: If the collection already exists and get_or_create is False.
- ValueError: If the collection name is invalid.

get_collection

Get a collection with the given name.

- `name` (str, required): The name of the collection to get
- `embedding_function` (Optional[EmbeddingFunction[Optional[Embeddings]]]): Optional function to use to embed documents. Uses the default embedding function if not provided.
- `data_loader` (Optional[DataLoader[Optional[Embeddings]]]): Optional function to use to load records (documents, images, etc.)

Returns: The collection

Raises:

- ValueError: If the collection does not exist

get_or_create_collection

Get or create a collection with the given name and metadata.

Args: name: The name of the collection to get or create metadata: Optional metadata to associate with the collection. If the collection already exists, the metadata provided is ignored. If the collection does not exist, the new collection will be created with the provided metadata. embedding_function: Optional function to use to embed documents data_loader: Optional function to use to load records (documents, images, etc.)

Returns: The collection

Examples:

```
client.get_or_create_collection("my_collection")
# collection(name="my_collection", metadata={})
```

delete_collection

Delete a collection with the given name.

- `name` (str, required): The name of the collection to delete.

Raises:

- ValueError: If the collection does not exist.

reset

Resets the database. This will delete all collections and entries.

Returns: True if the database was reset successfully.

get_version

Get the version of Chroma.

Returns: The version of Chroma

get_settings

Get the settings used to initialize.

Returns: The settings used to initialize.

get_max_batch_size

Return the maximum number of records that can be created or mutated in a single call.

Admin Client Methods

create_tenant

Create a new tenant. Raises an error if the tenant already exists.

get_tenant

Get a tenant. Raises an error if the tenant does not exist.

create_database

Create a new database. Raises an error if the database already exists.

get_database

Get a database. Raises an error if the database does not exist.

- `tenant` (str): The tenant of the database to get.

delete_database

Delete a database. Raises an error if the database does not exist.

- `tenant` (str): The tenant of the database to delete.

list_databases

List all databases for a tenant. Raises an error if the tenant does not exist.

- `tenant` (str): The tenant to list databases for.

Collection

Source: https://docs.trychroma.com/reference/python/collection

Collection Methods

count

Return the number of records in the collection.

add

Add records to the collection.

- **`ids`** (Union[str, IDs], required): Record IDs to add.
- **`embeddings`** (Optional[Embeddings]): Embeddings to add. If None, embeddings are computed.
- **`metadatas`** (Union[Optional[Metadatas], List[Optional[Metadatas]], None]): Optional metadata for each record.
- **`documents`** (Union[str, IDs, None]): Optional documents for each record.
- **`images`** (Optional[Embeddings]): Optional images for each record.
- **`uris`** (Union[str, IDs, None]): Optional URIs for loading images.

Raises:

- ValueError: If embeddings and documents are both missing.
- ValueError: If embeddings and documents are both provided.
- ValueError: If lengths of provided fields do not match.
- ValueError: If an ID already exists.

get

Retrieve records from the collection.

If no filters are provided, returns records up to `limit` starting at `offset`.

- **`ids`** (Union[str, IDs, None]): If provided, only return records with these IDs.
- **`where`** (Optional[Dict[Union[str, Literal[$and], Literal[$or]], Where]]): A Where filter used to filter based on metadata values.
- **`limit`** (Optional[int]): Maximum number of results to return.
- **`offset`** (Optional[int]): Number of results to skip before returning.
- **`where_document`** (Optional[Dict[Where, Union[str, List[Dict[Where, Union[str, List[WhereDocument]]]]]]]): A WhereDocument filter used to filter based on K.DOCUMENT.
- **`include`** (List[Literal[documents, embeddings, metadatas, distances, uris, data]]): Fields to include in results. Can contain "embeddings", "metadatas", "documents", "uris". Defaults to

"metadatas" and "documents".

Returns: Retrieved records and requested fields as a GetResult object.

peek

Return the first `limit` records from the collection.

- **`limit`** (int): Maximum number of records to return.

Returns: Retrieved records and requested fields.

query

Query for the K nearest neighbor records in the collection.

This is a batch query API. Multiple queries can be performed at once by providing multiple embeddings, texts, or images.

```
query_1 = [0.1, 0.2, 0.3]
query_2 = [0.4, 0.5, 0.6]
results = collection.query(
    query_embeddings=[query_1, query_2],
    n_results=10,
)
```

If query_texts, query_images, or query_uris are provided, the collection's embedding function will be used to create embeddings before querying the API.

The `ids`, `where`, `where_document`, and `include` parameters are applied to all queries.

- **`query_embeddings`** (Optional[Embeddings]): Raw embeddings to query for.
- **`query_texts`** (Union[str, IDs, None]): Documents to embed and query against.
- **`query_images`** (Optional[Embeddings]): Images to embed and query against.
- **`query_uris`** (Union[str, IDs, None]): URIs to be loaded and embedded.
- **`ids`** (Union[str, IDs, None]): Optional subset of IDs to search within.
- **`n_results`** (int): Number of neighbors to return per query.
- **`where`** (Optional[Dict[Union[str, Literal[$and], Literal[$or]], Where]]): Metadata filter.
- **`where_document`** (Optional[Dict[Where, Union[str, List[Dict[Where, Union[str, List[WhereDocument]]]]]]]): Document content filter.
- **`include`** (List[Literal[documents, embeddings, metadatas, distances, uris, data]]): Fields to include in results. Can contain "embeddings", "metadatas", "documents", "uris", "distances". Defaults to "metadatas", "documents", "distances".

Returns: Nearest neighbor results.

Raises:

- ValueError: If no query input is provided.

- ValueError: If multiple query input types are provided.

modify

Update collection name, metadata, or configuration.

- `name` (Optional[str]): New collection name.
- `metadata` (Optional[Dict[str, Any]]): New metadata for the collection.
- `configuration` (Optional[UpdateCollectionConfiguration]): New configuration for the collection.

update

Update existing records by ID.

Records are provided in columnar format. If provided, the `embeddings`, `metadatas`, `documents`, and `uris` lists must be the same length. Entries in each list correspond to the same record.

```
ids = ["id1", "id2", "id3"]
embeddings = [[0.1, 0.2, 0.3], [0.4, 0.5, 0.6], [0.7, 0.8, 0.9]]
metadatas = [{"key": "value"}, {"key": "value"}, {"key": "value"}]
documents = ["document1", "document2", "document3"]
uris = ["uri1", "uri2", "uri3"]
collection.update(ids, embeddings, metadatas, documents, uris)
```

If `embeddings` are not provided, the embeddings will be computed based on `documents` using the collection's embedding function.

- `ids` (Union[str, IDs], required): Record IDs to update.
- `embeddings` (Optional[Embeddings]): Updated embeddings. If None, embeddings are computed.
- `metadatas` (Union[Optional[Metadatas], List[Optional[Metadatas]], None]): Updated metadata.
- `documents` (Union[str, IDs, None]): Updated documents.
- `images` (Optional[Embeddings]): Updated images.
- `uris` (Union[str, IDs, None]): Updated URIs for loading images.

upsert

Create or update records by ID.

- `ids` (Union[str, IDs], required): Record IDs to upsert.
- `embeddings` (Optional[Embeddings]): Embeddings to add or update. If None, embeddings are computed.
- `metadatas` (Union[Optional[Metadatas], List[Optional[Metadatas]], None]): Metadata to add or update.
- `documents` (Union[str, IDs, None]): Documents to add or update.

- **`images`** (Optional[Embeddings]): Images to add or update.
- **`uris`** (Union[str, IDs, None]): URIs for loading images.

delete

Delete records by ID or filters.

All documents that match the `ids` or `where` and `where_document` filters will be deleted.

- **`ids`** (Optional[IDs]): Record IDs to delete.
- **`where`** (Optional[Dict[Union[str, Literal[$and], Literal[$or]], Where]]): Metadata filter.
- **`where_document`** (Optional[Dict[Where, Union[str, List[Dict[Where, Union[str, List[WhereDocument]]]]]]]): Document content filter.

Raises:

- ValueError: If no IDs or filters are provided.

Types

GetResult

Result payload for collection.get() operations.

The returned records are in columnar form. Corresponding entries in each list correspond to the same record.

```
results = collection.get(ids=["id1", "id2", "id3"])
records = zip(results["ids"], results["documents"], results["metadatas"])
for id, document, metadata in records:
    print(id, document, metadata)
```

GetResult will only include ids and the fields specified in the `include` param when making the get() operation.

Properties

QueryResult

Result payload for collection.query() operations.

The returned records are batches of records in columnar form.

```
results = collection.query(query_embeddings=[batch_1, batch_2, ...])
batches = zip(results["ids"], results["documents"], results["metadatas"])
```

Each batch is a list of records in columnar form.

```
for batch in batches:
    records = zip(batch["ids"], batch["documents"], batch["metadatas"])
    for id, document, metadata in records:
        print(id, document, metadata)
```

QueryResult will only include ids and the fields specified in the `include` param when making the query() operation.

Properties

Embedding Functions

Source: https://docs.trychroma.com/reference/python/embedding-functions

Embedding Function Base Classes

EmbeddingFunction

Protocol for embedding functions.

To implement a new embedding function, you need to implement the following methods: - **init** - **call** - name - build_from_config - get_config

Additionally, you should register the embedding function so it will automatically be used by the Chroma client.

```
@register_embedding_function
class MyEmbeddingFunction(EmbeddingFunction[Documents]):
    ...
```

Methods

`__init__()`, `build_from_config()`, `default_space()`, `embed_query()`, `embed_with_retries()`, `get_config()`, `is_legacy()`, `name()`, `supported_spaces()`, `validate_config()`, `validate_config_update()`

SparseEmbeddingFunction

Protocol for sparse embedding functions.

To implement a new sparse embedding function, you need to implement the following methods: - **call** - **init** - name - build_from_config - get_config

Methods

`__init__()`, `build_from_config()`, `embed_query()`, `embed_with_retries()`, `get_config()`, `name()`, `validate_config()`, `validate_config_update()`

Registration

register_embedding_function

Register a custom embedding function.

Can be used as a decorator:

```
@register_embedding_function
class MyEmbedding(EmbeddingFunction):
    @classmethod
    def name(cls): return "my_embedding"
```

Or directly:

```
register_embedding_function(MyEmbedding)
```

- **`ef_class`** (Any): The embedding function class to register.

register_sparse_embedding_function

Register a custom sparse embedding function.

Can be used as a decorator:

```
@register_sparse_embedding_function
class MySparseEmbeddingFunction(SparseEmbeddingFunction):
    @classmethod
    def name(cls): return "my_sparse_embedding"
```

Types

Embedding

`Embedding[Tuple[Any, Ellipsis], dtype[Union[int32, float32]]]`

SparseVector

Sparse vector using parallel indices and values arrays.

Properties

Methods

`__init__()`, `from_dict()`, `to_dict()`

Where Filters

Source: https://docs.trychroma.com/reference/python/where-filter

Use the `K` (Key) builder to construct where filters in Python. Filters are passed to `get`, `query`, `search`, `delete`, and similar methods via the `where` parameter.

Field references

Type	DSL	Example
Metadata field	`K("field_name")`	`K("category")`, `K("year")`
Document	`K.DOCUMENT`	`K.DOCUMENT.contains("text")`
ID	`K.ID`	`K.ID.is_in(["id1", "id2"])`

Comparison operators

Predicate	Operator	Example
Equal	`==`	`K("status") == "active"`
Not equal	`!=`	`K("count") != 0`
Greater than	`>`	`K("price") > 100`
Greater than or equal	`>=`	`K("year") >= 2020`
Less than	`<`	`K("stock") < 10`
Less than or equal	`<=`	`K("discount") <= 0.25`

Set operators

Predicate	DSL	Example
In list	`K("field").is_in([...])`	`K("category").is_in(["tech", "ai"])`
Not in list	`K("field").not_in([...])`	`K("status").not_in(["draft", "deleted"])`

Array operators

Predicate	DSL	Example
Contains	`K("field").contains(value)`	`K("tags").contains("action")`
Not contains	`K("field").not_contains(value)`	`K("tags").not_contains("draft")`

Document operators

Predicate	DSL	Example
Contains	`K.DOCUMENT.contains(value)`	`K.DOCUMENT.contains("machine learning")`
Not contains	`K.DOCUMENT.not_contains(value)`	`K.DOCUMENT.not_contains("draft")`

Predicate	DSL	Example
Regex match	`K.DOCUMENT.regex(pattern)`	`K.DOCUMENT.regex("^quantum\\s+\\w+")`
Regex not match	`K.DOCUMENT.not_regex(pattern)`	`K.DOCUMENT.not_regex("^draft")`

Troubleshooting

Source: https://docs.trychroma.com/docs/overview/troubleshooting

This page is a list of common gotchas or issues and how to fix them.

If you don't see your problem listed here, please also search the Github Issues.

Chroma JS-Client failures on NextJS projects

Our default embedding function uses @huggingface/transformers, which depends on binaries that NextJS fails to bundle. If you are running into this issue, you can wrap your `nextConfig` (in `next.config.ts`) with the `withChroma` plugin, which will add the required settings to overcome the bundling issues.

Cannot return the results in a contiguous 2D array. Probably ef or M is too small

This error happens when the HNSW index fails to retrieve the requested number of results for a query, given its structure and your data. he way to resolve this is to either decrease the number of results you request from a query (n_result), or increase the HNSW parameters `M`, `ef_construction`, and `ef_search`. You can read more about HNSW configurations here.

Using .get or .query, embeddings say `None`

This is actually not an error. Embeddings are quite large and heavy to send back. Most application don't use the underlying embeddings and so, by default, chroma does not send them back.

To send them back: add `include=["embeddings", "documents", "metadatas", "distances"]` to your query to return all information.

For example:

```
results = collection.query(
    query_texts="hello",
    n_results=1,
    include=["embeddings", "documents", "metadatas", "distances"],
)
```

> **Note:** We may change `None` to something else to more clearly communicate why they were not returned.

Build error when running `pip install chromadb`

If you encounter an error like this during setup

```
Failed to build hnswlib
ERROR: Could not build wheels for hnswlib, which is required to install pyproject.toml-based
projects
```

Try these few tips from the community:

1. If you get the error: `clang: error: the clang compiler does not support '-march=native'`, set this ENV variable, `export HNSWLIB_NO_NATIVE=1`
2. If on Mac, install/update xcode dev tools, `xcode-select --install`
3. If on Windows, try these steps

SQLite

Chroma requires SQLite > 3.35, if you encounter issues with having too low of a SQLite version please try the following.

1. Install the latest version of Python 3.10, sometimes lower versions of python are bundled with older versions of SQLite.
2. If you are on a Linux system, you can install pysqlite3-binary, `pip install pysqlite3-binary` and then override the default sqlite3 library before running Chroma with the steps here. Alternatively you can compile SQLite from scratch and replace the library in your python installation with the latest version as documented here.
3. If you are on Windows, you can manually download the latest version of SQLite from https://www.sqlite.org/download.html and replace the DLL in your python installation's DLLs folder with the latest version. You can find your python installation path by running `os.path.dirname(sys.executable)` in python.
4. If you are using a Debian based Docker container, older Debian versions do not have an up to date SQLite, please use `bookworm` or higher.

Illegal instruction (core dumped)

If you encounter an error like this during setup and are using Docker - you may have built the library on a machine with a different CPU architecture than the one you are running it on. Try rebuilding the Docker image on the machine you are running it on.

My data directory is too large

If you were using Chroma prior to v0.5.6, you may be able to significantly shrink your database by vacuuming it. After vacuuming once, automatic pruning (a new feature in v0.5.6) is enabled and will keep your database size in check.

Migration

Source: https://docs.trychroma.com/docs/overview/migration

Schema and data format changes are a necessary evil of evolving software. We take changes seriously and make them infrequently and only when necessary.

Chroma's commitment is whenever schema or data format change, we will provide a seamless and easy-to-use migration tool to move to the new schema/format.

Specifically we will announce schema changes on:

- Discord (#migrations channel)
- Github (here)
- Email listserv Sign up

We will aim to provide:

- a description of the change and the rationale for the change.
- a CLI migration tool you can run
- a video walkthrough of using the tool

Migration Log

v1.0.0 - March 1, 2025

In this release, we've rewritten much of Chroma in Rust. Performance has significantly improved across the board.

Breaking changes

Chroma no longer provides built-in authentication implementations.

`list_collections` now reverts back to returning `Collection` objects.

Chroma in-process changes

This section is applicable to you if you use Chroma via

```
client = chromadb.Client()
# or
client = chromadb.EphemeralClient()
# or
client = chromadb.PersistentClient()
```

The new Rust implementation ignores these settings:

- `chroma_server_nofile`
- `chroma_server_thread_pool_size`
- `chroma_memory_limit_bytes`
- `chroma_segment_cache_policy`

Chroma CLI changes

This section is applicable to you if you run a Chroma server using the CLI (`chroma run`).

Settings that you may have previously provided to the server using environment variables, like `CHROMA_SERVER_CORS_ALLOW_ORIGINS` or `CHROMA_OTEL_COLLECTION_ENDPOINT`, are now provided using a configuration file. For example:

```
chroma run --config ./config.yaml
```

Check out a full sample configuration file here.

Chroma in Docker changes

This section is applicable to you if you run Chroma using a Docker container.

Settings that you previously provided to the container using environment variables, like `CHROMA_SERVER_CORS_ALLOW_ORIGINS` or `CHROMA_OTEL_COLLECTION_ENDPOINT`, are now provided to the container using a configuration file. See the Docker documentation for more information.

The default data location in the container has changed from `/chroma/chroma` to `/data`. For example, if you previously started the container with:

```
docker run -p 8000:8000 -v ./chroma:/chroma/chroma chroma-core/chroma
```

you should now start it with:

```
docker run -p 8000:8000 -v ./chroma:/data chroma-core/chroma
```

v0.6.0 - December 30, 2024

Previously, `list_collections` returned a list of `Collection` objects. This could lead to some errors if any of your collections were created with a custom embedding function (i.e. not the default). So moving forward, `list_collections` will only return collections names.

For example, if you created all your collections with the `OpenAIEmbeddingFunction` , this is how you will use `list_collections` and `get_collection` correctly:

```
collection_names = client.list_collections()
ef = OpenAIEmbeddingFunction(...)
collections = [
    client.get_collection(name=name, embedding_function=ef)
    for name in collection_names
]
```

In the future, we plan on supporting embedding function persistence, so `list_collections` can return properly configured `Collection` objects, and you won't need to supply the correct embedding function to `get_collection`.

Additionally, we have dropped support for Python 3.8

v0.5.17 - October 30, 2024

We no longer support sending empty lists or dictionaries for metadata filtering, ID filtering, etc. For example,

```
collection.get(
    ids=["id1", "id2", "id3", ...],
```

```
    where={}
)
```

is not supported. Instead, use:

```
collection.get(ids=["id1", "id2", "id3", ...])
```

v0.5.12 - October 8, 2024

The operators `$ne` (not equal) and `$nin` (not in) in `where` clauses have been updated:

- Previously: They only matched records that had the specified key.
- Now: They also match records that don't have the specified key at all.

In other words, `$ne` and `$nin` now match the complement set of records (the exact opposite) that `$eq` (equals) and `$in` (in) would match, respectively.

The `$not_contains` operator in the `where_document` clause has also been updated:

- Previously: It only matched records that had a document field.
- Now: It also matches records that don't have a document field at all.

In other words, `$not_contains` now matches the exact opposite set of records that `$contains` would match.

`RateLimitingProvider` is now deprecated and replaced by `RateLimitEnforcer`. This new interface allows you to wrap server calls with rate limiting logic. The default `SimpleRateLimitEnforcer` implementation allows all requests, but you can create custom implementations for more advanced rate limiting strategies.

v0.5.11 - September 26, 2024

The results returned by `collection.get()` is now ordered by internal ids. Whereas previously, the results were ordered by user provided ids, although this behavior was not explicitly documented. We would like to make the change because using user provided ids may not be ideal for performance in hosted Chroma, and we hope to propagate the change to local Chroma for consistency of behavior. In general, newer documents in Chroma has larger internal ids.

A subsequent change in behavior is `limit` and `offset`, which depends on the order of returned results. For example, if you have a collection named `coll` of documents with ids `["3", "2", "1", "0"]` inserted in this order, then previously `coll.get(limit=2, offset=2)["ids"]` gives you `["2", "3"]`, while currently this will give you `["1", "0"]`.

We have also modified the behavior of `client.get_or_create`. Previously, if a collection already existed and the `metadata` argument was provided, the existing collection's metadata would be overwritten with the new values. This has now changed. If the collection already exists, get_or_create will simply return the existing collection with the specified name, and any additional arguments-including `metadata`-will be ignored.

Finally, the embeddings returned from `collection.get()`, `collection.query()`, and `collection.peek()` are now represented as 2-dimensional NumPy arrays instead of Python lists. When adding embeddings, you can still use either a Python list or a NumPy array. If your request returns multiple embeddings, the result will be a Python list containing 2-dimensional NumPy

arrays. This change is part of our effort to enhance performance in Local Chroma by using NumPy arrays for internal representation of embeddings.

v0.5.6 - September 16, 2024

Chroma internally uses a write-ahead log. In all versions prior to v0.5.6, this log was never pruned. This resulted in the data directory being much larger than it needed to be, as well as the directory size not decreasing by the expected amount after deleting a collection.

In v0.5.6 the write-ahead log is pruned automatically. However, this is not enabled by default for existing databases. After upgrading, you should run `chroma utils vacuum` once to reduce your database size and enable continuous pruning. See the CLI reference for more details.

This does not need to be run regularly and does not need to be run on new databases created with v0.5.6 or later.

v0.5.1 - June 7, 2024

On the Python client, the `max_batch_size` property was removed. It wasn't previously documented, but if you were reading it, you should now use `get_max_batch_size()`.

The first time this is run, it makes a HTTP request. We made this a method to make it more clear that it's potentially a blocking operation.

Auth overhaul - April 20, 2024

If you are not using Chroma's built-in auth system, you do not need to take any action.

This release overhauls and simplifies our authentication and authorization systems. If you are you using Chroma's built-in auth system, you will need to update your configuration and any code you wrote to implement your own authentication or authorization providers. This change is mostly to pay down some of Chroma's technical debt and make future changes easier, but it also changes and simplifies user configuration. If you are not using Chroma's built-in auth system, you do not need to take any action.

Previously, Chroma's authentication and authorization relied on many objects with many configuration options, including:

- `chroma_server_auth_provider`
- `chroma_server_auth_configuration_provider`
- `chroma_server_auth_credentials_provider`
- `chroma_client_auth_credentials_provider`
- `chroma_client_auth_protocol_adapter`

and others.

We have consolidated these into three classes:

- `ClientAuthProvider`
- `ServerAuthenticationProvider`
- `ServerAuthorizationProvider`

`ClientAuthProvider`s are now responsible for their own configuration and credential management. Credentials can be given to them with the `chroma_client_auth_credentials` setting. The value for

`chroma_client_auth_credentials` depends on the `ServerAuthenticationProvider`; for `TokenAuthenticationServerProvider` it should just be the token, and for `BasicAuthenticationServerProvider` it should be `username:password`.

`ServerAuthenticationProvider`s are responsible for turning a request's authorization information into a `UserIdentity` containing any information necessary to make an authorization decision. They are now responsible for their own configuration and credential management. Configured via the `chroma_server_authn_credentials` and `chroma_server_authn_credentials_file` settings.

`ServerAuthorizationProvider`s are responsible for turning information about the request and the `UserIdentity` which issued the request into an authorization decision. Configured via the `chroma_server_authz_config` and `chroma_server_authz_config_file` settings.

Either `_authn_credentials` or `authn_credentials_file` can be set, never both. Same for `authz_config` and `authz_config_file`. The value of the config (or data in the config file) will depend on your authn and authz providers. See here for more information.

The two auth systems Chroma ships with are `Basic` and `Token`. We have a small migration guide for each.

Basic

If you're using `Token` auth, your server configuration might look like:

```
CHROMA_SERVER_AUTH_CREDENTIALS="admin:admin"
CHROMA_SERVER_AUTH_CREDENTIALS_FILE="./example_file"
CHROMA_SERVER_AUTH_CREDENTIALS_PROVIDER="chromadb.auth.providers.HtpasswdConfigurationServerA
uthCredentialsProvider"
CHROMA_SERVER_AUTH_PROVIDER="chromadb.auth.basic.BasicAuthServerProvider"
```

Note: Only one of `AUTH_CREDENTIALS` and `AUTH_CREDENTIALS_FILE` can be set, but this guide shows how to migrate both.

And your corresponding client configation:

```
CHROMA_CLIENT_AUTH_PROVIDER="chromadb.auth.token.TokenAuthClientProvider"
CHROMA_CLIENT_AUTH_CREDENTIALS="admin:admin"
```

To migrate to the new server configuration, simply change it to:

```
CHROMA_SERVER_AUTHN_PROVIDER="chromadb.auth.token_authn.TokenAuthenticationServerProvider"
CHROMA_SERVER_AUTHN_CREDENTIALS="test-token"
CHROMA_SERVER_AUTHN_CREDENTIALS_FILE="./example_file"
```

New client configuration:

```
CHROMA_CLIENT_AUTH_CREDENTIALS="test-token"
CHROMA_CLIENT_AUTH_PROVIDER="chromadb.auth.basic_authn.BasicAuthClientProvider"
```

Token

If you're using `Token` auth, your server configuration might look like:

```
CHROMA_SERVER_AUTH_CREDENTIALS="test-token"
CHROMA_SERVER_AUTH_CREDENTIALS_FILE="./example_file"
CHROMA_SERVER_AUTH_CREDENTIALS_PROVIDER="chromadb.auth.token.TokenConfigServerAuthCredentials
Provider"
CHROMA_SERVER_AUTH_PROVIDER="chromadb.auth.token.TokenAuthServerProvider"
CHROMA_SERVER_AUTH_TOKEN_TRANSPORT_HEADER="AUTHORIZATION"
```

Note: Only one of `AUTH_CREDENTIALS` and `AUTH_CREDENTIALS_FILE` can be set, but this guide shows how to migrate both.

And your corresponding client configation:

```
CHROMA_CLIENT_AUTH_PROVIDER="chromadb.auth.token.TokenAuthClientProvider"
CHROMA_CLIENT_AUTH_CREDENTIALS="test-token"
CHROMA_CLIENT_AUTH_TOKEN_TRANSPORT_HEADER="AUTHORIZATION"
```

To migrate to the new server configuration, simply change it to:

```
CHROMA_SERVER_AUTHN_PROVIDER="chromadb.auth.token_authn.TokenAuthenticationServerProvider"
CHROMA_SERVER_AUTHN_CREDENTIALS="test-token"
CHROMA_SERVER_AUTHN_CREDENTIALS_FILE="./example_file"
CHROMA_AUTH_TOKEN_TRANSPORT_HEADER="AUTHORIZATION"
```

New client configuration:

```
CHROMA_CLIENT_AUTH_CREDENTIALS="test-token"
CHROMA_CLIENT_AUTH_PROVIDER="chromadb.auth.token_authn.TokenAuthClientProvider"
CHROMA_AUTH_TOKEN_TRANSPORT_HEADER="AUTHORIZATION"
```

Reference of changed configuration values

- Overall config
- `chroma_client_auth_token_transport_header`: renamed to `chroma_auth_token_transport_header`.
- `chroma_server_auth_token_transport_header`: renamed to `chroma_auth_token_transport_header`.
- Client config
- `chroma_client_auth_credentials_provider`: deleted. Functionality is now in `chroma_client_auth_provider`.
- `chroma_client_auth_protocol_adapter`: deleted. Functionality is now in `chroma_client_auth_provider`.
- `chroma_client_auth_credentials_file`: deleted. Functionality is now in `chroma_client_auth_credentials`.
- These changes also apply to the Typescript client.
- Server authn
- `chroma_server_auth_provider`: Renamed to `chroma_server_authn_provider`.
- `chroma_server_auth_configuration_provider`: deleted. Functionality is now in `chroma_server_authn_provider`.
- `chroma_server_auth_credentials_provider`: deleted. Functionality is now in `chroma_server_authn_provider`.
- `chroma_server_auth_credentials_file`: renamed to `chroma_server_authn_credentials_file`.
- `chroma_server_auth_credentials`: renamed to `chroma_server_authn_credentials`.
- `chroma_server_auth_configuration_file`: renamed to `chroma_server_authn_configuration_file`.
- Server authz
- `chroma_server_authz_ignore_paths`: deleted. Functionality is now in `chroma_server_auth_ignore_paths`.

To see the full changes, you can read the PR or reach out to the Chroma team on Discord.

Migration to 0.4.16 - November 7, 2023

This release adds support for multi-modal embeddings, with an accompanying change to the definitions of `EmbeddingFunction`. This change mainly affects users who have implemented their own `EmbeddingFunction` classes. If you are using Chroma's built-in embedding functions, you do not need to take any action.

EmbeddingFunction

Previously, `EmbeddingFunction`s were defined as:

```
class EmbeddingFunction(Protocol):
    def __call__(self, texts: Documents) -> Embeddings:
        ...
```

After this update, `EmbeddingFunction`s are defined as:

```
Embeddable = Union[Documents, Images]
D = TypeVar("D", bound=Embeddable, contravariant=True)

class EmbeddingFunction(Protocol[D]):
    def __call__(self, input: D) -> Embeddings:
        ...
```

The key differences are:

- `EmbeddingFunction` is now generic, and takes a type parameter `D` which is a subtype of `Embeddable`. This allows us to define `EmbeddingFunction`s which can embed multiple modalities.
- `__call__` now takes a single argument, `input`, to support data of any type `D`. The `texts` argument has been removed.

Migration from >0.4.0 to 0.4.0 - July 17, 2023

What's new in this version?

- New easy way to create clients
- Changed storage method
- `.persist()` removed, `.reset()` no longer on by default

New Clients

```
### in-memory ephemeral client

# before

client = chromadb.Client()

# after

client = chromadb.EphemeralClient()


### persistent client

# before
```

```
from chromadb.config import Settings
client = chromadb.Client(Settings(
    chroma_db_impl="duckdb+parquet",
    persist_directory="/path/to/persist/directory" # Optional, defaults to .chromadb/ in the
current directory
))

# after

client = chromadb.PersistentClient(path="/path/to/persist/directory")


### http client (to talk to server backend)

# before

from chromadb.config import Settings
client = chromadb.Client(Settings(chroma_api_impl="rest",
                                        chroma_server_host="localhost",
                                        chroma_server_http_port="8000"
                                    ))

# after

client = chromadb.HttpClient(host="localhost", port="8000")
```

You can still also access the underlying `.Client()` method. If you want to turn off telemetry, all clients support custom settings:

```
from chromadb.config import Settings
client = chromadb.PersistentClient(
    path="/path/to/persist/directory",
    settings=Settings(anonymized_telemetry=False))
```

New data layout

This version of Chroma drops `duckdb` and `clickhouse` in favor of `sqlite` for metadata storage. This means migrating data over. We have created a migration CLI utility to do this.

If you upgrade to `0.4.0` and try to access data stored in the old way, you will see this error message

> You are using a deprecated configuration of Chroma. Please pip install chroma-migrate and run `chroma-migrate` to upgrade your configuration. See https://docs.trychroma.com/deployment/migration for more information or join our discord at https://discord.gg/MMeYNTmh3x for help!

Here is how to install and use the CLI:

```
pip install chroma-migrate
chroma-migrate
```

If you need any help with this migration, please reach out! We are on Discord ready to help.

Persist & Reset

`.persist()` was in the old version of Chroma because writes were only flushed when forced to. Chroma `0.4.0` saves all writes to disk instantly and so `persist` is no longer needed.

`.reset()`, which resets the entire database, used to by enabled-by-default which felt wrong. `0.4.0` has it disabled-by-default. You can enable it again by passing `allow_reset=True` to a Settings object. For example:

```
from chromadb.config import Settings
client = chromadb.PersistentClient(path="./path/to/chroma",
settings=Settings(allow_reset=True))
```